SIMATS ENGINEERING
Engineer to Excel

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DATABASE MANAGEMENT SYSTEMS (CSA0501)

Assignment 1 - CO1

Design, Analyse and Validate a Storage and Indexing Strategy for the Museum Condition-Report Database

Case: Meridian Museum - Collection Loans and Exhibition Planning

Name: SK ALI BABU

Register No.: 192524038

Submission date: 31/10/2026

Total marks: 50 Nature: Individual Duration: 1 week

How this report is organised

The report follows the twelve tasks in the assignment sheet in order. Tasks 1 to 7 are design work (requirements, keys, relationships, ER, EER, weak entities and mapping). Tasks 8 and 9 are the executed part: the schema was created, filled with generated data at four sizes, tested with invalid writes and checked with eight queries. Tasks 10 and 11 compare models and defend the final choice. Task 12 handles the changed requirement (shared ownership with changing shares).

Every number shown as "actual output" in this report was produced by running the scripts, not typed by hand. The same random seed (4211) was used for all four population sizes, so the runs can be repeated and will give the same counts.

Execution environment (please read)

The queries and tests were executed on **SQLite 3.45 through Python** because a MySQL server was not available in the environment where this work was prepared. The schema uses only ordinary SQL (primary keys, foreign keys, UNIQUE, CHECK), and the MySQL 8.0 version of the DDL is given in Appendix A. Two things differ when it is run on MySQL: the wording of the error messages (for example MySQL reports "ERROR 1062 Duplicate entry" and "ERROR 3819 Check constraint is violated") and the exact millisecond timings. The result counts will be identical because the data generator and seed are the same. If the course requires screenshots from MySQL Workbench, run Appendix A and the queries in Task 9 and replace the timing table.

0. Case Analysis and Assumptions

Before designing anything I listed every statement in the scenario and noted what it forces on the design. This table is used later to trace requirements to entities and keys (Task 11).

Table 1. Case statements and their design consequences

Statement in the case	What it forces in the design
8 collection units, 35 curators, 1,200 lenders, 9,000 artifacts	Reference data (units) is small; lenders and artifacts are the large tables. Scale ratio used for generated data: 7.5 artifacts per lender (9,000 / 1,200).
Each artifact has one current lender	Original assumption A1: ARTIFACT carries a single current lender. This is the assumption that breaks in Task 12.
A loan belongs to one lender and contains multiple artifacts	LOAN is 1:N to loan lines; the lender is stored once on LOAN, not repeated per artifact.
An artifact can have successive loans	LOAN and ARTIFACT are M:N, so the line between them needs its own identity (LOAN_OBJECT). Successive custody periods of one artifact must not overlap.
Display slots are numbered within an exhibition	Slot number repeats across exhibitions, so DISPLAY_SLOT is a weak entity identified by (exhibition, slot_no).
A curator may lend a privately owned object	Lender and employee roles overlap on the same real person, so one PERSON identity is needed.
Same artifact has different descriptions in loan register and gallery plan	One ARTIFACT record is the only place where the description lives. Every other record stores artifact_id.
Loan expiry is not linked to display slots	Placement must point to the loan line, so the loan end date can be compared with the exhibition dates.
Condition reports are filed by lender name	A report must belong to a physical object in a specific loan (LOAN_OBJECT), not to a lender.

Statement in the case	What it forces in the design
Ownership, custody, placement and condition evidence must be separate	Four separate concepts: ARTIFACT_OWNER (Task 12), LOAN/LOAN_OBJECT (custody), PLACEMENT (display), CONDITION_REPORT (evidence).

Assumptions I made

- "Today" is 31 October 2026 when deciding whether a loan is overdue.
- A loan is returned as a whole (one returned_date per loan). If objects came back on different days, returned_date would move to LOAN_OBJECT.
- Condition grades are A, B, C and D. Registrars file the reports. A curator handling a loan must not be the lender of the same loan (conflict of interest).
- The three concepts that must never be mixed: **ownership** (who legally owns the object and in what share), **custody** (which loan has the object and for which dates) and **placement** (which display slot it occupies in an exhibition).

Task 1 - Requirement Analysis and Alternative Conceptual Designs

5 marks

1.1 Operational users who create LOAN records

Table 2. Operational users and shared information

User	What they do with a loan	Information they must share
Loan administrator (lender office)	Creates the LOAN record when terms are agreed; adds one line per artifact	loan_id, lender, handling curator, start date, end date, list of artifacts, loan value per artifact
Curator	Proposes the loan and plans the exhibition	Artifact identity (catalog_no), exhibition dates, loan end date
Registrar	Receives and returns the objects; files condition reports	loan_id + artifact_id for every report, grade, report date
Gallery planner	Assigns display slots	Artifact identity, loan end date (must cover the exhibition), slot number

What the four groups share is the pair **(loan, artifact)**. If that pair is not a record of its own, each group keeps its own copy, which is exactly the "register versus gallery plan" mismatch described in the case.

1.2 Three alternative conceptual designs

The alternatives differ in three ways the assignment names: how the loan occurrence is identified (identity), where the history is kept (historical representation), and who the people are (identity of persons). They are not just three names for the same boxes.

Alternative A - dated binary relationship (history lives on the relationship)

LENDER and ARTIFACT are joined by an M:N relationship LENDS that carries start_date and end_date. There is no loan record. CURATOR and LENDER are separate entities. Condition reports are filed against the lender, using the lender name as the identifier, which is what the museum does today.

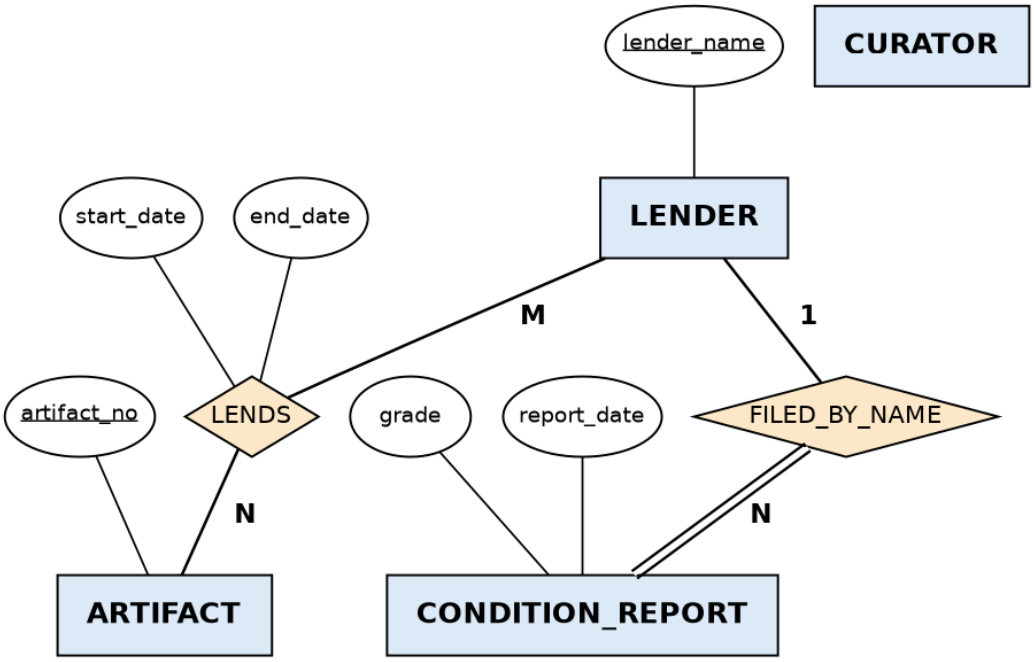


Figure 1. Alternative A: dated relationship LENDS, condition report filed by lender name

- **Who creates it:** the registrar enters one LENDS row per artifact. The loan administrator has no loan number to quote, so the office and the gallery cannot refer to "the same loan".
- **Dated relationship vs independent LOAN:** here a loan occurrence is identified only by (lender, artifact, start_date). It cannot be referred to by other relationships.
- **Separate LENDER and employee records:** a curator who lends a private object exists twice (as CURATOR and as LENDER) with two phone numbers.
- **Custody and evidence:** custody is only two dates on the relationship. A condition report can point to the lender but not to one particular lending of one particular object.
- **Failure it is meant to resolve:** the loan register and the gallery plan describe the same artifact differently. In A both documents reference ARTIFACT by key. **Its own weakness:** Raman Trust lends MM-010 in Sep-Dec 2025 and again in Sep-Dec 2026. A report that says "Raman Trust - scratch" cannot say which year or which object.

Alternative B - independent LOAN occurrence with LOAN_OBJECT (identity of the loan)

LOAN is an entity with its own key. It is made by one lender, handled by one curator and contains lines (LOAN_OBJECT) that point to ARTIFACT. Condition reports belong to the loan line. LENDER and CURATOR are still separate entities.

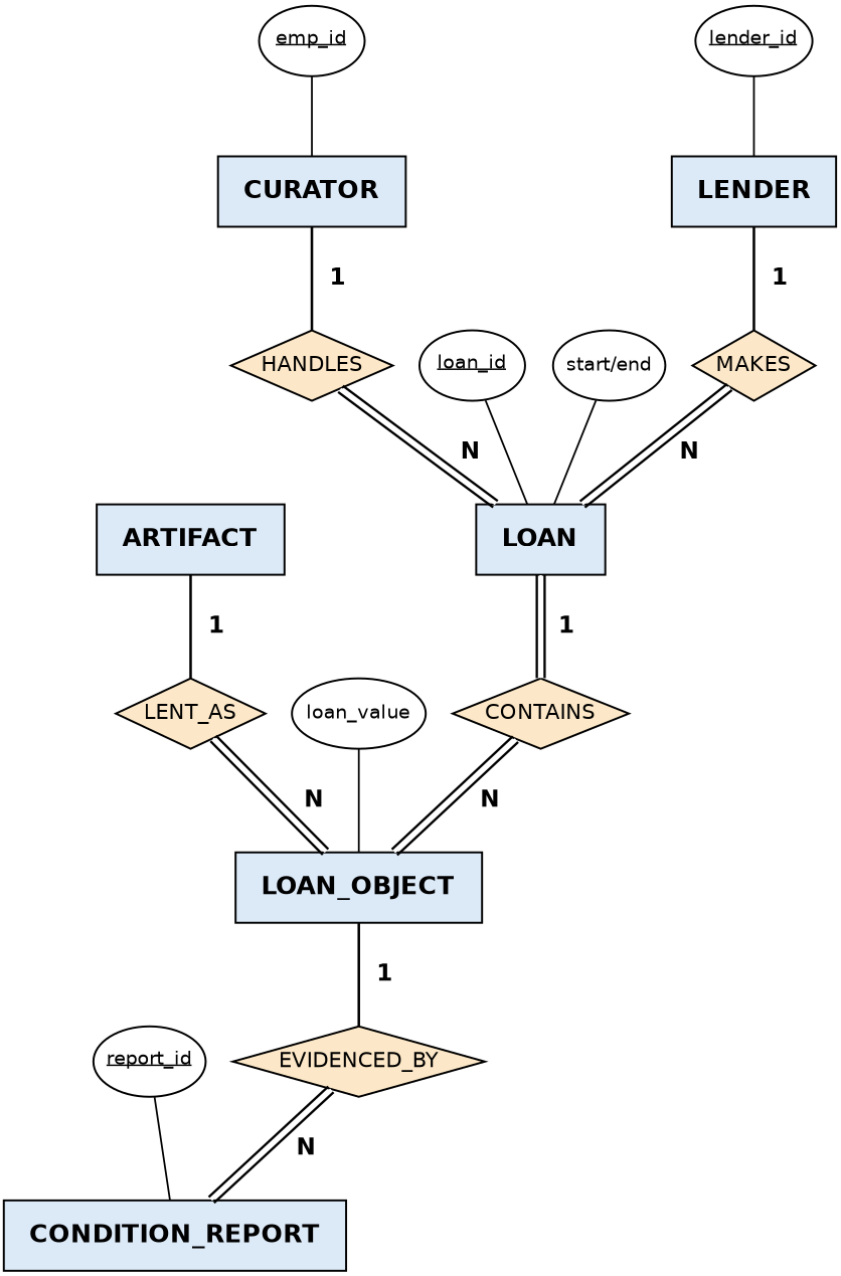


Figure 2. Alternative B: independent LOAN and LOAN_OBJECT, evidence owned by the loan line

- **Who creates it:** the loan administrator creates LOAN once; the registrar and planner both reference loan_id and artifact_id.
- **Dated relationship vs independent LOAN:** the dates are attributes of LOAN, and LOAN can take part in CONTAINS, HANDLES, MAKES. A relationship cannot do this in basic ER.
- **Custody and evidence:** each custody period is one LOAN row. A report points to (loan_id, artifact_id), so it is tied to one object in one period.

- **Failure it is meant to resolve:** "condition reports are filed by lender name". Report on MM-011 in loan 100 stays with (100, 11) and cannot be confused with MM-010 in loan 100 or with MM-011 in an earlier loan of the same lender.
- **Remaining weakness:** LENDER and CURATOR are separate record types, so one real person can still exist twice.

Alternative C - B plus one shared PERSON identity (identity of people)

A single PERSON entity holds name, e-mail and phones. LENDER and EMPLOYEE are roles of PERSON (overlapping), and CURATOR/REGISTRAR are employee roles. LOAN references the person in the lender role and the person in the curator role. The artifact still has one current lender at this stage.

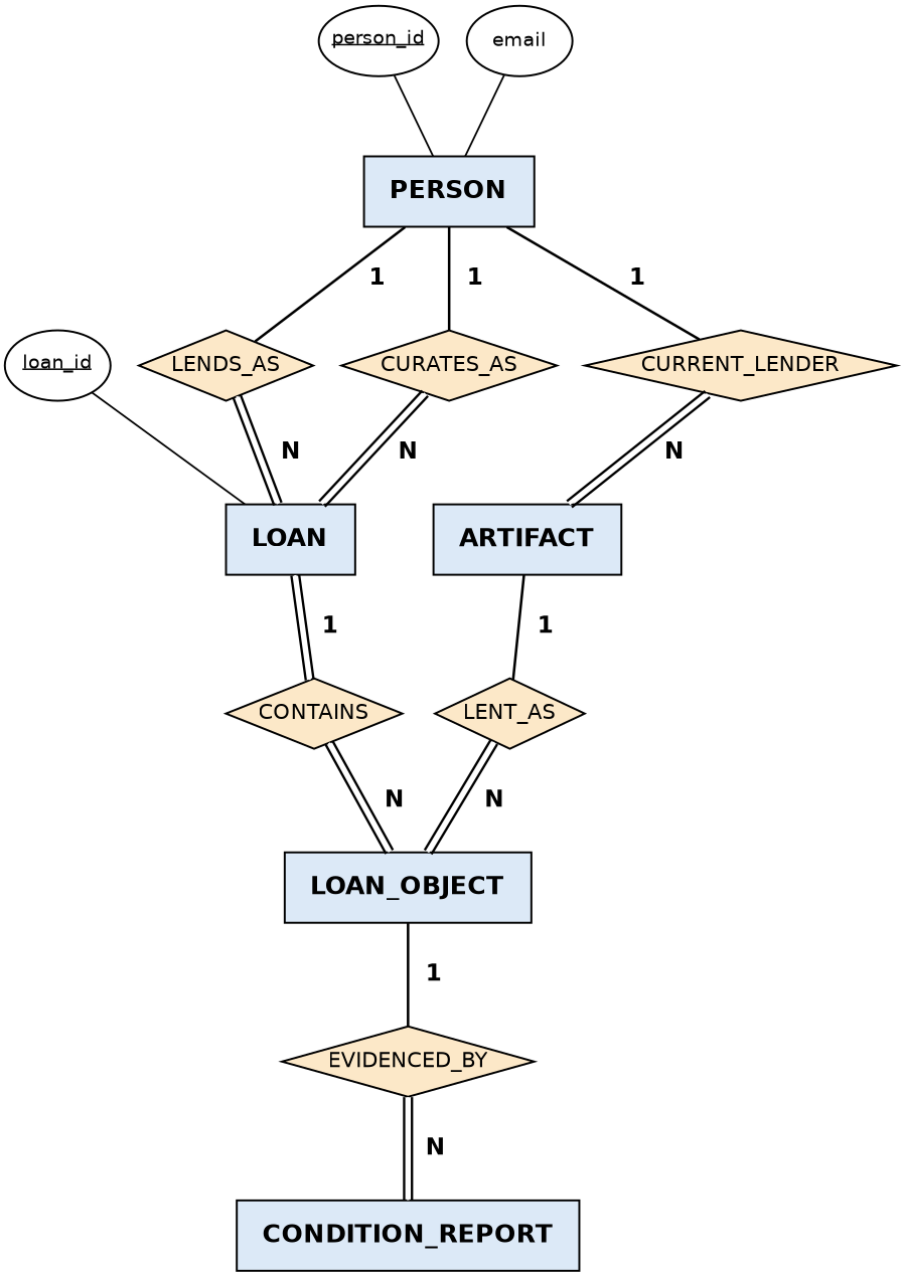


Figure 3. Alternative C: shared PERSON identity, independent LOAN, single current lender per artifact

- **Who creates it:** same users as B. The difference is that the lender office searches one person register, so a curator who also lends is found, not re-typed.
- **Shared PERSON vs separate records:** with separate records, curator Nair at 1 phone and lender Nair at another cannot be known to be the same human. With PERSON there is one phone list.
- **Custody and evidence:** same as B. In addition, the rule "the handling curator is not the lender" can be checked by comparing two person ids.

- **Failure it is meant to resolve:** a curator lends a private vase and also handles that loan. In B the two identities cannot be compared; in C lender_id = curator_id exposes it.

1.3 Why renaming LOAN would not change the model

Suppose Alternative A simply renames the relationship LENDS to LOAN. Nothing changes: its identity is still the participants plus a date, it still cannot be referenced by CONDITION_REPORT or PLACEMENT, and it still has no place to say "these two artifacts belong to the same loan".

Counterexample from this case. Lender 1017 sends artifacts 11 and 14 on 1 September 2026 under one agreement with curator Divya. In A the museum stores two LOAN rows (1017, 11, 2026-09-01) and (1017, 14, 2026-09-01). The curator and the end date are repeated, and nothing records that the two rows are one loan. If the same lender sends artifact 11 again on the same day under a second agreement, the key (1017, 11, 2026-09-01) collides. An independent LOAN entity solves both: loan 100 holds both artifacts and loan 101 is a separate occurrence.

Table 3. Comparison of the three alternatives

Feature	A: dated relationship	B: independent LOAN	C: B + shared PERSON
Loan identity	(lender, artifact, start_date)	loan_id (own key)	loan_id (own key)
Relationship degree	Binary M:N with attributes	Binary 1:N chains, LOAN_OBJECT as associative entity	Same as B
History of custody	Dates on the relationship	One LOAN row per custody period	One LOAN row per custody period
Condition evidence owned by	Lender (by name)	LOAN_OBJECT	LOAN_OBJECT
Person identity	Separate LENDER and CURATOR	Separate LENDER and CURATOR	One PERSON, roles overlap

Feature	A: dated relationship	B: independent LOAN	C: B + shared PERSON
Main failure it resolves	Register and gallery plan use different descriptions	Reports filed by lender name; loan expiry cannot reach slots	Curator-lender duplicated identity; conflict of interest
Main weakness	No loan identity; no evidence per object	Duplicate people	Needs role constraints (Task 5)

Choice: I carry Alternative C forward. Task 3 and Task 4 draw it, Task 5 adds the EER detail, and Task 11 gives the full justification. The ownership question is left for Task 12, which is why Alternative C still has a single current lender on ARTIFACT.

Task 2 - Entities, Attributes and Keys

2 marks

2.1 Entity and key catalog

Table 4. Entity and key catalog

Entity	Kind	Primary key	Candidate / business keys	Reason
PERSON	Strong, independent	person_id (surrogate)	email	Real-world person; name is not unique so it cannot be a key.
LENDER (role)	Subtype of PERSON	person_id	-	Shares PERSON identity.
EMPLOYEE (role)	Subtype of PERSON	person_id	-	Role CURATOR / REGISTRAR / OTHER in emp_role.
COLLECTION_UNIT	Strong	unit_id	unit_name	8 units; has its own data.
ARTIFACT	Strong	artifact_id	catalog_no	Physical object; the only place its description is stored.

Entity	Kind	Primary key	Candidate / business keys	Reason
LOAN	Strong (independent occurrence)	loan_id	none natural	(lender, start_date) is not unique: one lender can open two loans the same day.
LOAN_OBJECT	Associative entity	(loan_id, artifact_id)	-	One artifact appears once per loan.
CONDITION_REPORT	Dependent on LOAN_OBJECT	report_id	-	Several reports allowed per loan line (arrival, departure).
EXHIBITION	Strong	exhibition_id	-	Names repeat across editions, so name is not a key.
DISPLAY_SLOT	Weak, existence-dependent	(exhibition_id, slot_no)	-	slot_no is only unique inside one exhibition.
PLACEMENT	Association record	(loan_id, artifact_id)	(exhibition_id, slot_no)	An object has at most one slot and a slot holds at most one object.

Entity	Kind	Primary key	Candidate / business keys	Reason
ORGANIZATION / NOTIFICATION_CONTACT	Strong / category	org_id / contact_id	org_name	Contact can be a person or an organization (Task 5).

Independently identifiable: PERSON, ARTIFACT, LOAN, EXHIBITION, COLLECTION_UNIT, CONDITION_REPORT. **Existence-dependent:** DISPLAY_SLOT (cannot exist without its exhibition), LOAN_OBJECT (cannot exist without its loan), PLACEMENT.

2.2 Attribute classification

Table 5. Attribute classification

Attribute	Class	Treatment
email, lender_type, emp_role, grade, title, catalog_no	Simple, single-valued	One column each.
Contact: phone	Multivalued	Separate table person_phone (person_id, phone_no).
Contact: person name	Composite	Stored as given_name and family_name.
CollectionType (collection unit of an artifact)	Simple coded attribute	Promoted to entity COLLECTION_UNIT because it has its own data (name) and relationships (artifacts, curators).
start_date, end_date, returned_date, report_date, open_date, close_date	Simple, stored (source dates)	Stored as DATE. They are facts entered by people, not calculated.

Attribute	Class	Treatment
duration_days, overdue flag, objects per loan, total loan value, report count	Derived (calculated summaries)	Not stored. Computed in views from source dates and lines, so they cannot go out of date.

2.3 Key selection and justification

- **Stable primary keys:** system-issued integers (person_id, artifact_id, loan_id, report_id, exhibition_id). They never change, even when a name, a catalog number or a lender changes.
- **Business candidate keys:** person.email, artifact.catalog_no, collection_unit.unit_name are declared UNIQUE. They are used to look things up, not to link tables, because they may be corrected later.
- **Owner-scoped key for DISPLAY_SLOT:** (exhibition_id, slot_no). Slot 7 exists in every exhibition, so slot_no alone identifies nothing.

Why an exhibition name cannot identify a display slot

Meridian can run "Bronze Age Treasures" in 2025 and again in 2027, and an exhibition can be renamed after it opens. "Bronze Age Treasures / slot 7" would then point to two different physical places, or to none after a rename. The numeric exhibition_id never changes, so the key (exhibition_id, slot_no) stays valid.

Why a person name is not a reliable identifier

I checked this on the generated data. In the 100-lender population 74 of 140 persons share their full name with someone else. In the 1,000-lender population it is 1,036 of 1,040, and the most common name ("Asha Khan") occurs 12 times. The generator draws names from short lists on purpose so this problem is visible, but real museums have the same problem with common names. A name also changes after marriage or for a trust that is renamed.

How concurrent registration avoids duplicate identifiers

Two registrars may register a new artifact at the same moment. If the application did SELECT MAX(artifact_id)+1, both would read the same maximum and both would try to insert the same number. The DBMS must issue the numbers instead.

Table 6. Race on identifier generation

Time	Registrar 1	Registrar 2	Result with MAX()+1	Result with AUTO_INCREMENT
t1	reads max = 9000			
t2		reads max = 9000		
t3	inserts 9001		9001 stored	gets 9001 from the counter
t4		inserts 9001	duplicate key error	gets 9002 from the counter

With an AUTO_INCREMENT column (MySQL) or a sequence the counter is advanced atomically inside the engine, so each insert receives a different number. The PRIMARY KEY and the UNIQUE keys are the last line of defence: if a duplicate ever reached the table it would be rejected (see the failure tests in Task 8).

Task 3 - Relationships and Structural Constraints

6 marks

3.1 Binary associations or a ternary LENDER-CURATOR-EXHIBITION?

A ternary relationship COORDINATES(LENDER, CURATOR, EXHIBITION) looks attractive because a lender, a curator and an exhibition are all involved in a loan. To test whether it is really a three-way fact, I built a small instance and projected it to the three binary relationships.

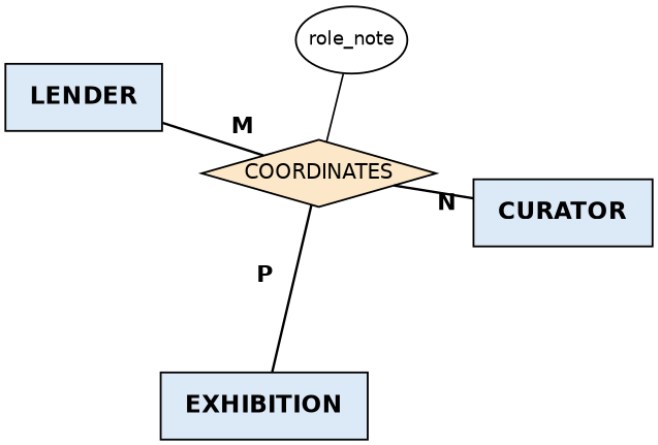


Figure 4. Possible ternary association

Table 7. Ternary instance

Ternary instance T	Lender	Curator	Exhibition
t1	Raman	Divya	E1
t2	Raman	Karthik	E2
t3	Asha	Divya	E2

Binary projections: LC = {(Raman, Divya), (Raman, Karthik), (Asha, Divya)}, CE = {(Divya, E1), (Karthik, E2), (Divya, E2)}, LE = {(Raman, E1), (Raman, E2), (Asha, E2)}.

Join LC with CE on curator, then with LE on (lender, exhibition). The result has four tuples: t1, t2, t3 and the extra tuple **(Raman, Divya, E2)**. That tuple was never in T. It says Divya handled a Raman loan in E2, which never happened, and it would wrongly create a custody period and a condition-report duty. So the three-way fact cannot be rebuilt from binary relationships, and the ternary relationship cannot simply be split.

What I use instead: the three-way fact is not a fact in itself in Meridian. It is the result of an independent LOAN. Each loan has exactly one lender and one handling curator, and its objects are placed in slots of exhibitions. With loan_id as the join key the spurious tuple cannot appear: loans 100 (Raman, Divya, E1), 101 (Raman, Karthik, E2), 102 (Asha, Divya, E2) join back to exactly three tuples, because loan_id is a key of every projection. So the model keeps binary associations and lets LOAN carry the connection.

3.2 Compulsory links and parents without dependents

Table 8. Compulsory and optional participation

Link	Compulsory?	Can the parent exist without dependent records?
LOAN to LENDER	Yes, exactly one	A lender can exist with no loan (new lender).
LOAN to CURATOR	Yes, exactly one	A curator can exist with no loan.
LOAN to LOAN_OBJECT	Yes, at least one line	A loan with no object has no meaning, so LOAN cannot exist without a line.
LOAN_OBJECT to ARTIFACT	Yes, exactly one	An artifact can exist with no loan (not lent yet).
LOAN_OBJECT to CONDITION_REPORT	Report must have its line; line need not have a report	Yes. The line exists first, the report is filed after inspection.
LOAN_OBJECT to DISPLAY_SLOT	Optional both sides	Yes. An object can be in storage and a slot can be empty.
DISPLAY_SLOT to EXHIBITION	Yes, identifying	An exhibition can be created before its slots.

Effect of missing LOAN_OBJECT evidence. I deliberately allow a loan line with zero reports (minimum 0). If a report were compulsory for the line (minimum 1), neither row could be created first: the report needs the line, and the line would need a report. Missing evidence is therefore handled by a query (Task 9, Q2) and a workflow rule, not by the structure.

3.3 Participation bounds for six relationships

Notation (min, max) is written next to the entity it describes: how many relationship instances one entity occurrence takes part in.

Table 9. Min/max participation bounds

Relationship	Left entity (min,max)	Right entity (min,max)	Defence from the case
LENDER - MAKES - LOAN	LENDER (0,N)	LOAN (1,1)	A loan belongs to one lender; a new lender may have no loan yet.
CURATOR - HANDLES - LOAN	CURATOR (0,N)	LOAN (1,1)	Every loan has one responsible curator; custody periods are traceable to a person.
LOAN - CONTAINS - LOAN_OBJECT	LOAN (1,N)	LOAN_OBJECT (1,1)	A loan contains multiple artifacts; a line belongs to one loan.
ARTIFACT - LENT_AS - LOAN_OBJECT	ARTIFACT (0,N)	LOAN_OBJECT (1,1)	Successive loans give many lines per artifact, one per custody period.
LOAN_OBJECT - EVIDENCED_BY - CONDITION_REPORT	LOAN_OBJECT (0,N)	CONDITION_REPORT (1,1)	Evidence belongs to one object in one period; several reports are possible (arrival and return).

Relationship	Left entity (min,max)	Right entity (min,max)	Defence from the case
EXHIBITION - HAS_SLOT - DISPLAY_SLOT	EXHIBITION (0,N)	DISPLAY_SLOT (1,1)	Slots are numbered within one exhibition and die with it.
LOAN_OBJECT - PLACED_IN - DISPLAY_SLOT	LOAN_OBJECT (0,1)	DISPLAY_SLOT (0,1)	An object occupies at most one slot; a slot holds at most one object.

3.4 Where dated and quantitative facts belong

Table 10. Placement of dated and quantitative facts

Fact	Belongs on	Reason
start_date, end_date, returned_date	LOAN	They describe the whole agreement and are the same for every artifact in it.
loan_value (insured value for this loan)	LOAN_OBJECT	It depends on both the loan and the artifact. It is different from the artifact catalog value.
report_date, grade	CONDITION_REPORT	Facts about one inspection.
Exhibition open/close dates	EXHIBITION	The slot has no dates of its own; the exhibition dates apply.
share_pct, valid_from, valid_to (Task 12)	ARTIFACT_OWNER association	They depend on artifact and person together and on time.

3.5 The composite placement-to-slot constraint

PLACEMENT stores (exhibition_id, slot_no) and these two columns together are a foreign key to DISPLAY_SLOT(exhibition_id, slot_no). It has to be composite, because slot_no alone does not name a slot. A single-column reference to slot_no would accept slot 7 of the

wrong exhibition. A UNIQUE constraint on (exhibition_id, slot_no) in PLACEMENT then makes sure a slot holds at most one object. Both behaviours were tested; see the failure tests in Task 8.

Task 4 - Chen ER Diagram

4 marks

The diagram shows the preferred model (Alternative C) before the ownership change. Two separate paths are drawn from LOAN_OBJECT: the path for borrowed objects (LOAN to LOAN_OBJECT to ARTIFACT, then to a display slot) and the path for condition reports (LOAN_OBJECT to CONDITION_REPORT, filed by a REGISTRAR).

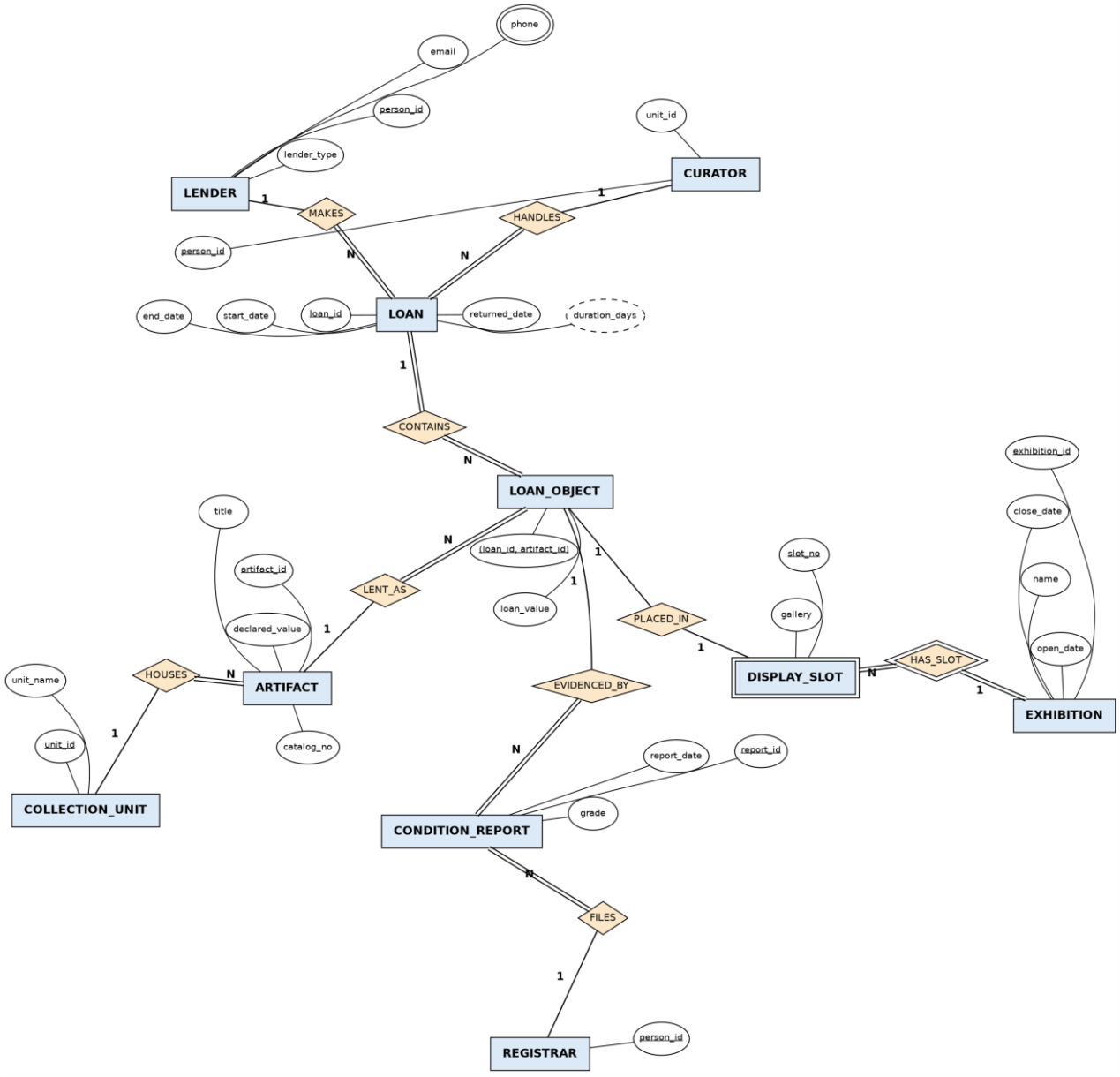


Figure 5. Chen ER diagram of the loan coordination model

Legend

Table 11. Legend for the ER diagram

Symbol	Meaning
Rectangle	Entity
Double rectangle	Weak entity (DISPLAY_SLOT)
Diamond / double diamond	Relationship / identifying relationship (HAS_SLOT)
Ellipse / double ellipse / dashed ellipse	Attribute / multivalued attribute / derived attribute
Underlined / dashed-underlined attribute	Primary key / partial key of a weak entity
Double line	Total (compulsory) participation
1, N on a line	Cardinality on that side of the relationship

Owner of DISPLAY_SLOT and its owner-scoped key

EXHIBITION is the identifying owner of DISPLAY_SLOT through the double diamond HAS_SLOT. slot_no is the partial key (dashed underline) and the complete key is (exhibition_id, slot_no).

Complete attribute catalog

Table 12. Attribute catalog reconciled with Task 2

Entity	Attributes (key underlined in diagram)	Notes
PERSON / LENDER / CURATOR / REGISTRAR	person_id, given_name, family_name, email, {phone}, lender_type, emp_role, unit_id	Shown as role boxes; shared attributes are drawn once in the EER of Task 5.
COLLECTION_UNIT	unit_id, unit_name	

Entity	Attributes (key underlined in diagram)	Notes
ARTIFACT	artifact_id, catalog_no, title, declared_value	Current lender is the relationship to PERSON in the original design.
LOAN	loan_id, start_date, end_date, returned_date, /duration_days	duration_days is derived.
LOAN_OBJECT	(loan_id, artifact_id), loan_value	Associative entity.
CONDITION_REPORT	report_id, report_date, grade	Filed by a registrar.
EXHIBITION	exhibition_id, name, open_date, close_date	
DISPLAY_SLOT (weak)	slot_no (partial key), gallery	Key is (exhibition_id, slot_no).

Reconciliation and repeated references

Every entity in the Task 2 catalog appears in the diagram or in the EER figure. The diagram keeps LENDER, CURATOR and REGISTRAR as separate boxes only so that the loan and report paths stay readable; their common supertype PERSON, with its name, e-mail and phone attributes, is drawn once in the EER (Task 5) instead of being repeated three times here. For the same reason ARTIFACT and LOAN are drawn again as repeated references in the EER of Task 12; they are the same entities, only reused to show the ownership addition. The custody and condition-evidence cardinalities annotated on the lines are the ones defended in Task 3: a loan has at least one line (double line on CONTAINS), a report must have its line (double line on EVIDENCED_BY) but a line may have no report (single line on the LOAN_OBJECT side).

Task 5 - Extending the Model to an EER Model

3 marks

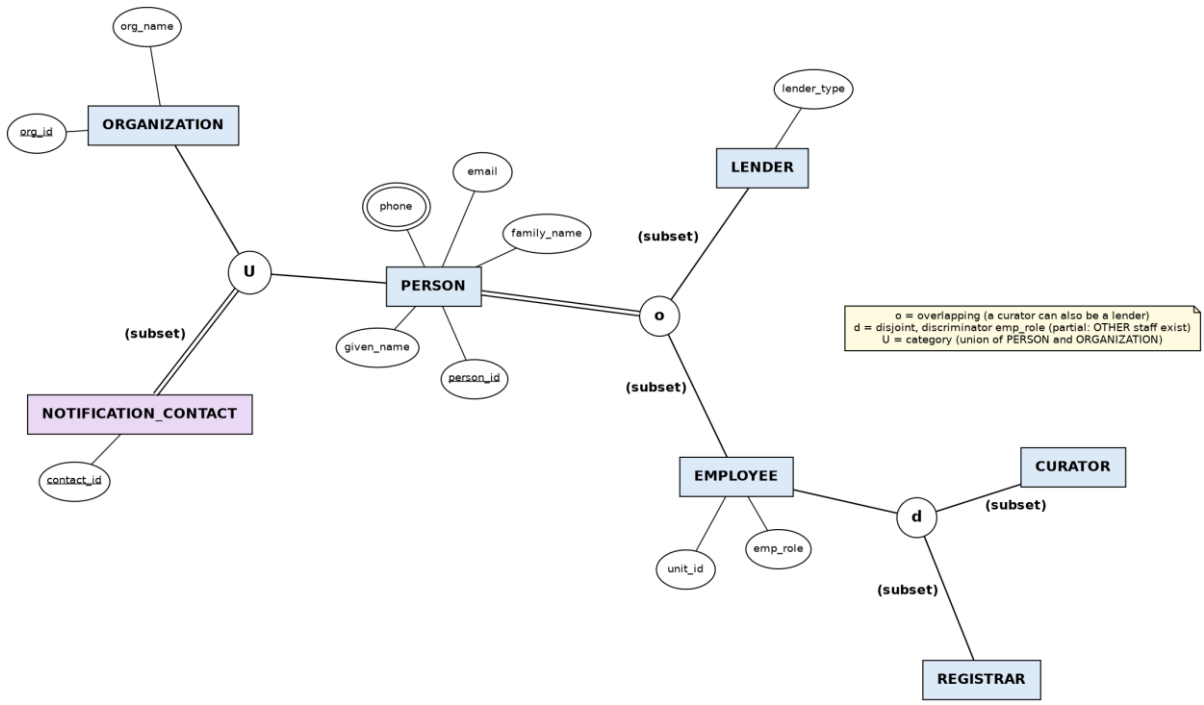


Figure 6. EER diagram: PERSON generalisation, employee specialisation and the notification-contact category

5.1 Generalisation and specialisation

LENDER and EMPLOYEE share name, e-mail and phone, so I generalised them into **PERSON**. Employees are then specialised into **CURATOR** and **REGISTRAR**. Having one PERSON means a curator who lends a private collection is one record with two roles, not two unrelated records.

5.2 Overlap and coverage

- **LENDER / EMPLOYEE is overlapping (o).** A curator can be a lender at the same time. This is real in the generated data: query Q8 in Task 9 returns 3 curators who also lend.
- **PERSON to LENDER/EMPLOYEE is total.** Every person in the register is at least a lender or an employee (the person_kind column records which: LENDER, EMPLOYEE or BOTH).
- **CURATOR / REGISTRAR is disjoint (d) and partial.** One employee has one emp_role. The specialisation does not cover all employees because other staff (role OTHER) exist who are neither.

5.3 How the discriminator controls membership

emp_role is the discriminator. A row with emp_role = CURATOR is a curator and may handle loans; a row with REGISTRAR may file condition reports. Because the value is single, the subtypes cannot overlap. The link to custody and evidence is direct: LOAN.curator_id should point only to curators, and CONDITION_REPORT.registrar_id only to registrars. The column CHECK limits emp_role to three allowed values, but it cannot check the role of the row that a foreign key points to. That is a cross-table rule and I test it in Task 8 (probe 1).

5.4 Notification contact as a category (union)

When a loan is overdue the museum notifies a contact, and that contact can be either a person (the lender or an agent) or an organization (a shipping or fine-art agent). PERSON and ORGANIZATION have different keys and attributes, so neither is a subtype of the other. NOTIFICATION_CONTACT is therefore a **category**: each contact is exactly one of PERSON or ORGANIZATION. It gets its own surrogate contact_id, and a row-level CHECK makes sure exactly one of person_id and org_id is filled. This can be enforced with a CHECK and it was tested (Task 8).

5.5 Subtype design compared with one wide record

Table 13. Subtype design versus one wide record

Question	Subtype design (PERSON + roles)	One wide record
Columns used by a typical row	Only the columns of its roles	Lender, curator and registrar columns in every row; most are NULL
Role-specific NOT NULL (unit_id for curators)	Possible in the role table	Impossible; the column must allow NULL for everyone else
LOAN.lender_id must be a lender	Foreign key to LENDER only	Foreign key to the wide table accepts a registrar
Curator who also lends	One PERSON row, one row in each role	One row, but flags and nullable columns grow

Question	Subtype design (PERSON + roles)	One wide record
Condition evidence and custody	Registrar and curator references are narrow and clear	A report could reference any person
Cost	A join to PERSON to read the name	No join, but wide rows and unclear meaning of NULL

Task 6 - Weak Entities and Complex Attributes

3 marks

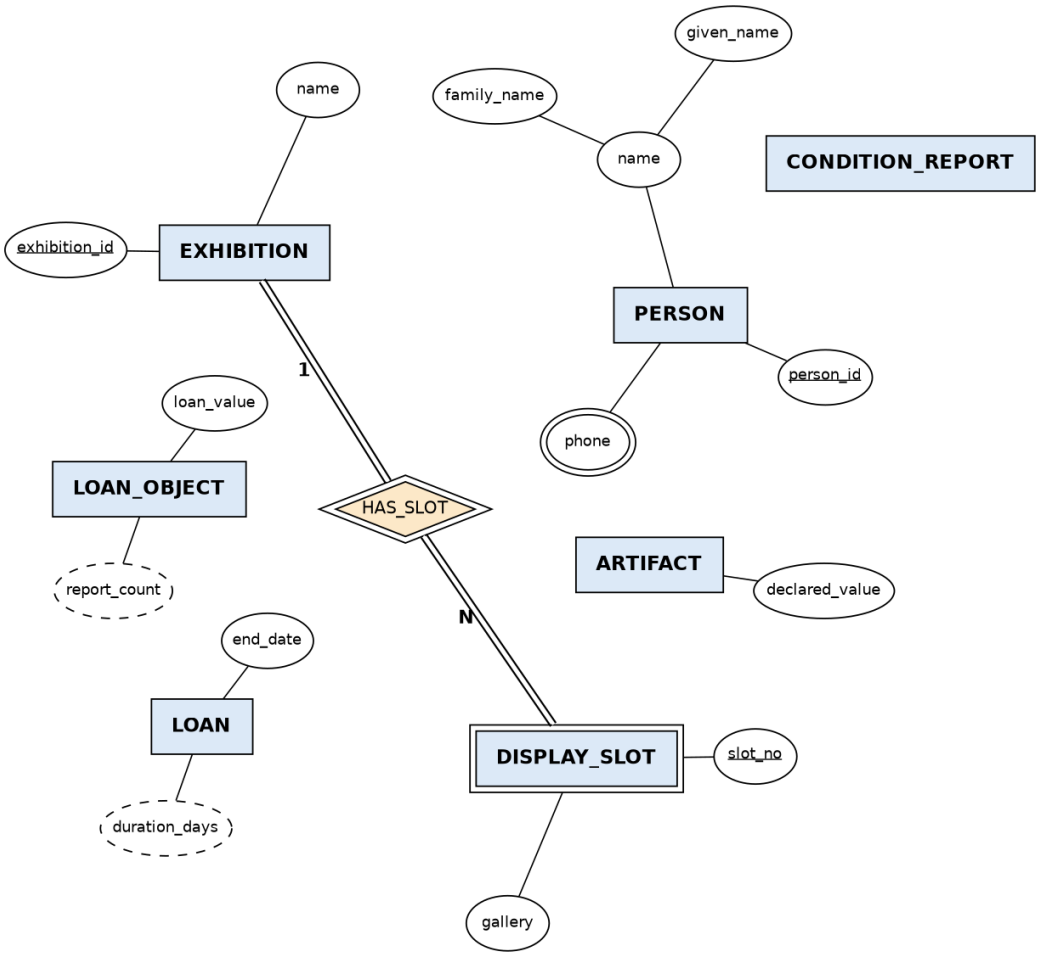


Figure 7. Weak entity DISPLAY_SLOT and complex attributes

6.1 Owner and local key of DISPLAY_SLOT

DISPLAY_SLOT is owned by EXHIBITION. Its local key is slot_no and its full key is (exhibition_id, slot_no). The link to the exhibition is an **identifying** link: exhibition_id is part of the primary key, the slot cannot be moved to another exhibition and it is deleted with its exhibition (ON DELETE CASCADE). An **ordinary** foreign key, such as LOAN.lender_id, is just a reference. It is not part of the key of LOAN, the loan keeps its own identity, and deleting the lender is blocked rather than cascaded.

6.2 Multiple telephone numbers

Phones are multivalued, so they go in their own table person_phone with primary key (person_id, phone_no). A comma-separated column would break first normal form: you could not search a number, you could not enforce that the same number is not stored twice for

one person, and changing one number would mean rewriting the string. In the generated data about 30% of lenders have a second number; at 100,000 lenders the table has 130,056 rows for 100,040 persons, and the museum still finds every number with an index lookup. For custody and evidence this matters because overdue notices (Q1) need the lender contact: with a separate table the notice can list every number.

6.3 Name components and computed summaries

The person name is a composite attribute, stored as given_name and family_name, so lists can be sorted by family name and a salutation can be built from the right part. The following values should be **computed, not stored**, because a stored copy would go out of date when the underlying rows change (an update anomaly):

- duration_days = end_date - start_date (derived from LOAN dates)
- overdue flag = returned_date is NULL and end_date is before today
- objects per loan, total loan value and report count per loan (derived from LOAN_OBJECT and CONDITION_REPORT)

Actual output of the view v_loan_summary on the 100-lender database (the summary is computed at read time):

```
CREATE VIEW v_loan_summary AS
SELECT l.loan_id, l.lender_id,
    CAST(julianday(l.end_date)-julianday(l.start_date) AS INTEGER) AS duration_days,
    COUNT(lo.artifact_id) AS objects, ROUND(SUM(lo.loan_value),2) AS
total_loan_value,
    (SELECT COUNT(*) FROM condition_report cr WHERE cr.loan_id=l.loan_id) AS
reports
FROM loan l JOIN loan_object lo ON lo.loan_id=l.loan_id GROUP BY l.loan_id;

SELECT * FROM v_loan_summary ORDER BY loan_id LIMIT 5;
```

Table 14. Output of v_loan_summary (first 5 loans, N = 100)

loan_id	lender_id	duration_days	objects	total_loan_value	reports
1	1084	162	3	173507.43	3
2	1061	162	2	80771.49	2

loan_id	lender_id	duration_days	objects	total_loan_value	reports
3	1045	160	4	186007.25	2
4	1067	157	3	213701.94	3
5	1028	153	2	33802.24	2

6.4 Which coded concepts need an entity

Table 15. Coded concepts and identity

Coded concept	Decision	Reason
Collection unit (CollectionType)	Entity COLLECTION_UNIT	Has its own data (name) and its own relationships (artifacts and curators belong to it).
Exhibition	Entity	Has dates, slots and placements.
Condition grade A-D	Column with CHECK	Four values, no extra data and no relationships.
lender_type, emp_role, person_kind	Column with CHECK	Short fixed lists with no data of their own.
Artifact value vs loan value	Two attributes in two places	ARTIFACT.declared_value is the catalog value. LOAN_OBJECT.loan_value is the insured value for one loan line and can differ per loan.

Task 7 - Mapping the EER Model to a Relational Schema

4 marks

7.1 Step-by-step mapping

Table 16. Mapping steps

Step	Rule applied	Result
1	Strong entities become relations with their key	person, collection_unit, artifact, loan, exhibition, condition_report, organization
2	Weak entity: owner key + partial key	display_slot(exhibition_id, slot_no, gallery), PK = both, FK to exhibition ON DELETE CASCADE
3	1:N relationships: foreign key on the N side	loan.lender_id, loan.curator_id, artifact.unit_id, artifact.current_lender_id, condition_report.registrar_id, employee.unit_id
4	M:N relationship: relation with both keys	loan_object(loan_id, artifact_id, loan_value). Several artifacts in one loan are several rows with the same loan_id.
5	1:1 relationship partial on both sides	placement(loan_id, artifact_id, exhibition_id, slot_no). Separate table, not nullable columns on either side. UNIQUE (exhibition_id, slot_no) keeps it 1:1.
6	Multivalued attribute	person_phone(person_id, phone_no)
7	Specialisation	See 7.2
8	Category	notification_contact(contact_id, contact_type, person_id, org_id) with CHECK for exactly one source
9	Report evidence references the loan line	condition_report(loan_id, artifact_id) is a composite FK to loan_object

One loan with several artifacts, taken from the actual 100-lender data (loan 3, four artifacts):

Table 17. Rows of loan_object for one loan (N = 100)

loan_id	artifact_id	catalog_no	loan_value
3	129	MM-0000129	15103.56
3	482	MM-0000482	65605.47
3	638	MM-0000638	36719.75
3	652	MM-0000652	68578.47

7.2 Mapping the overlapping PERSON roles

There are four standard options for a specialisation: (8A) one relation for the supertype and one for each subtype, (8B) relations for the subtypes only, (8C) one relation with a type attribute, and (8D) one relation with a flag per subtype. I used a mix:

- **LENDER and EMPLOYEE: option 8A.** The roles overlap, so repeating the person in two subtype-only relations (8B) would store a curator-lender twice. A single table with a type attribute (8C) cannot hold two values at once, and flags (8D) leave many NULLs. With 8A the common data lives once in person and each role keeps its own columns.
- **CURATOR and REGISTRAR: option 8C inside employee.** They are disjoint and have no attributes of their own, so a discriminator column emp_role in employee is enough. A separate table for each would add joins and nothing else.

Effect on custody and evidence: loan.lender_id references lender(person_id), so only a lender can lend; loan.curator_id and condition_report.registrar_id reference employee(person_id). The role value is not enforced by the foreign key (limitation, Task 11).

7.3 Schema catalog

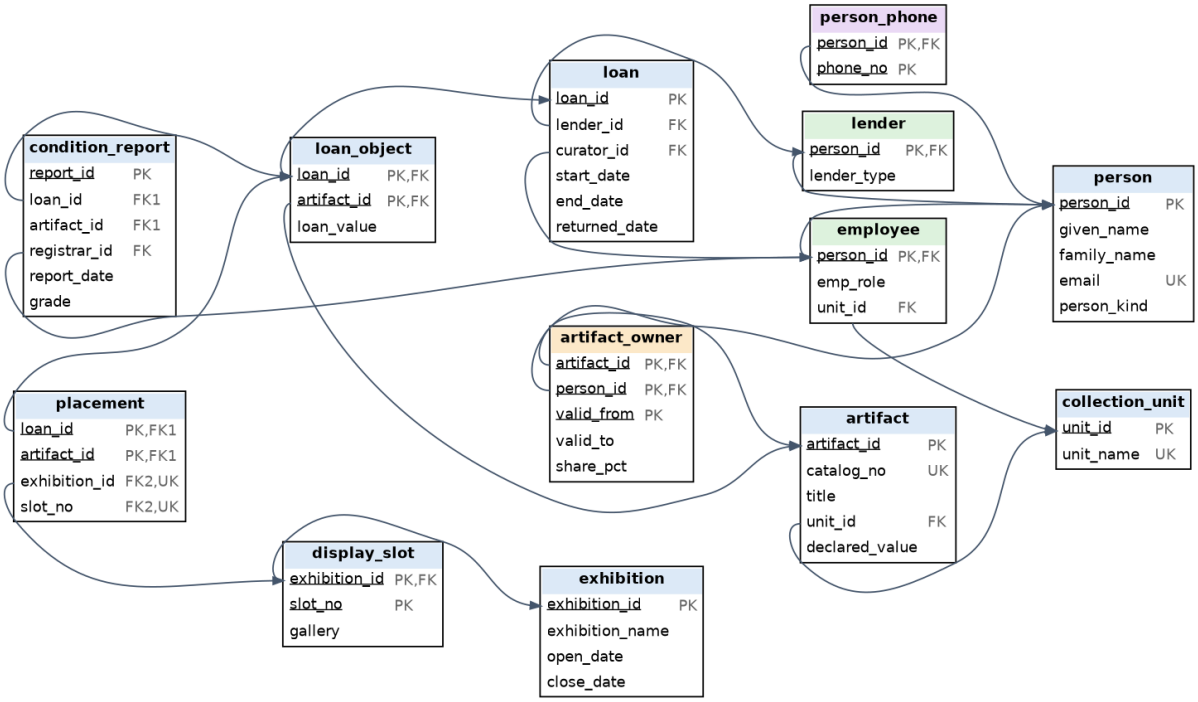


Figure 8. Relational schema (arrows run from the foreign key to the referenced key)

Table 18. Schema catalog

Relation	Primary key	Foreign keys	Deletion rule and reason
person	person_id	-	Never deleted while referenced.
person_phone	(person_id, phone_no)	person_id -> person	CASCADE; phones have no meaning without the person.
lender / employee	person_id	person_id -> person; employee.unit_id -> collection_unit	RESTRICT.
artifact	artifact_id	unit_id -> collection_unit; current_lender_id -> lender	RESTRICT.

Relation	Primary key	Foreign keys	Deletion rule and reason
loan	loan_id	lender_id -> lender; curator_id -> employee; notification_contact_id -> notification_contact	RESTRICT. Loan history is never deleted.
loan_object	(loan_id, artifact_id)	loan_id -> loan; artifact_id -> artifact	RESTRICT; custody history must survive.
condition_report	report_id	(loan_id, artifact_id) -> loan_object; registrar_id -> employee	RESTRICT; evidence is kept.
exhibition	exhibition_id	-	Deleted only through its slots (CASCADE on slots, blocked by placements).
display_slot	(exhibition_id, slot_no)	exhibition_id -> exhibition	CASCADE from exhibition.
placement	(loan_id, artifact_id)	(loan_id, artifact_id) -> loan_object; (exhibition_id, slot_no) -> display_slot; UNIQUE (exhibition_id, slot_no)	RESTRICT.
notification_contact / organization	contact_id / org_id	person_id -> person; org_id -> organization	RESTRICT.

Why deletion is restricted for historical LOAN records. A loan is evidence of who had which object and when. If a lender, artifact or loan line could be deleted, the condition reports and placements that depend on it would be left without context, or deleted by a cascade, and the museum would lose evidence it may need for an insurance claim. Corrections are made by updating the row (for example returned_date), not by deleting it. The delete test in Task 8 shows the DBMS refusing to delete a loan that still has lines.

Task 8 - Implementing the Schema and Populating Instances

8.1 Relations and constraints

The full MySQL 8.0 DDL is in Appendix A. The row-level checks that guard the loan period are:

> 123412

```
mysql> SELECT *
    -> FROM display_slot;
Empty set (0.005 sec)

mysql> SELECT * FROM exhibition;
Empty set (0.007 sec)

mysql> SELECT * FROM organization;
Empty set (0.005 sec)

mysql> SELECT * FROM notification_contact;
Empty set (0.005 sec)

mysql> SELECT * FROM loan;
Empty set (0.005 sec)

mysql> SELECT * FROM loan_object;
Empty set (0.005 sec)

mysql> SELECT * FROM condition_report;
Empty set (0.005 sec)

mysql> SELECT * FROM employee;
Empty set (0.005 sec)

mysql>
```

8.2 Reproducible loading procedure (seed 4211)

1. Create the empty schema from the DDL.
2. Set random.Random(4211) once per population size. The same seed is used for all four sizes, so each run is reproducible.
3. Insert reference data: 8 collection units, 40 employees (35 curators with ids 6-40 and 5 registrars with ids 1-5) and N lenders with ids from 1001. Three curators are also registered as lenders (role overlap).

4. Insert artifacts: 7.5 per lender (9,000 / 1,200). Each artifact receives a current lender drawn from the lender id list.
5. Create loans in four disjoint 6-month rounds. In each round a lender is picked at random among those that still have at least two unused artifacts, and the loan lines are taken from **that lender's own artifacts**, without replacement inside the round. So a loan line always points to an existing artifact owned by the loan's lender, and one artifact cannot be in two loans of the same round. Rounds do not overlap in time, so custody periods of the same artifact never overlap.
6. Condition reports: each loan line is selected with probability 0.85 and a report row is created that points to that existing (loan_id, artifact_id). The registrar is drawn from ids 1-5.
7. Exhibitions and slots are created per round with dates inside every loan period of that round (open day 20, close day 150 of a 180-day round); 60% of the loan lines get a free slot from a shuffled list, so no slot is used twice.
8. About 6% of loans that ended before 31 Oct 2026 are left without returned_date. These are the overdue open loans that Q1 should find.

How source keys are selected for linked rows: every child row copies a key from a parent row that has already been inserted (lender ids from the lender list, artifact ids from that lender's artifact list, loan lines from the loan list). The generator never invents a foreign key value, which is why foreign key checks never fail while loading. For custody and evidence this means a report always exists for a real object in a real loan.

```
mysql> SELECT 'collection_unit' AS table_name, COUNT(*) AS total_rows FROM collection_unit
    -> UNION ALL SELECT 'person', COUNT(*) FROM person
    -> UNION ALL SELECT 'person_phone', COUNT(*) FROM person_phone
    -> UNION ALL SELECT 'lender', COUNT(*) FROM lender
    -> UNION ALL SELECT 'employee', COUNT(*) FROM employee
    -> UNION ALL SELECT 'artifact', COUNT(*) FROM artifact
    -> UNION ALL SELECT 'organization', COUNT(*) FROM organization
    -> UNION ALL SELECT 'notification_contact', COUNT(*) FROM notification_contact
    -> UNION ALL SELECT 'loan', COUNT(*) FROM loan
    -> UNION ALL SELECT 'loan_object', COUNT(*) FROM loan_object
    -> UNION ALL SELECT 'condition_report', COUNT(*) FROM condition_report
    -> UNION ALL SELECT 'exhibition', COUNT(*) FROM exhibition
    -> UNION ALL SELECT 'display_slot', COUNT(*) FROM display_slot
    -> UNION ALL SELECT 'placement', COUNT(*) FROM placement;
+---------------------+------------+
| table_name          | total_rows |
+---------------------+------------+
| collection_unit     |          1 |
| person              |          1 |
| person_phone        |          0 |
| lender              |          0 |
| employee            |          0 |
| artifact            |          0 |
| organization        |          0 |
| notification_contact |          0 |
| loan                |          0 |
| loan_object         |          0 |
| condition_report    |          0 |
| exhibition          |          0 |
| display_slot        |          0 |
| placement           |          0 |
+---------------------+------------+
14 rows in set (0.053 sec)
```

```
mysql> SELECT person_id, email
    -> FROM person
    -> WHERE email = 'p1001@lender.example';
+-----------+----------------------+
| person_id | email                |
+-----------+----------------------+
|      9001 | p1001@lender.example |
+-----------+----------------------+
1 row in set (0.006 sec)
```

```
mysql> SELECT * FROM person;
+-----------+------------+-------------+----------------------+-------------+
| person_id | given_name | family_name | email                | person_kind |
+-----------+------------+-------------+----------------------+-------------+
|      9001 | Test       | User        | p1001@lender.example | LENDER      |
+-----------+------------+-------------+----------------------+-------------+
1 row in set (0.005 sec)
```

```
mysql> SELECT * FROM collection_unit;
+---------+-----------------+
| unit_id | unit_name       |
+---------+-----------------+
|       1 | Main Collection |
+---------+-----------------+
1 row in set (0.005 sec)
```

```
mysql> USE museum_db;
Database changed
mysql>
```

8.3 One stable schema, four changing populations

The schema stays the same. Only the number of lender and loan instances changes with the population size:

Table 19. Actual row counts after loading (seed 4211)

Table (rows)	N = 100	N = 1,000	N = 10,000	N = 100,000
collection_unit	8	8	8	8
employee	40	40	40	40
person	140	1,040	10,040	100,040
lender	103	1,003	10,003	100,003
artifact	750	7,500	75,000	750,000
loan	148	1,500	15,000	150,000
loan_object	423	4,426	44,214	442,363
condition_report	356	3,768	37,521	375,939
exhibition	12	73	669	6,641
display_slot	480	2,920	26,760	265,640
placement	253	2,654	26,526	265,416
organization	20	20	200	2,000
notification_contact	40	40	400	4,000

Loading time in this environment: 0 s, 0.2 s, 2 s and 28.1 s for the four sizes. The lender count is N + 3 because three curators are registered as lenders too, and the person count is N + 40 (the 40 employees).

8.4 Invalid writes and the actual failure messages

Each statement below was executed against the loaded 100-lender database with foreign keys enabled. The message is the one the engine returned (SQLite wording; MySQL words these differently).

Identity writes

Attempt	Statement	Actual message
Identity: duplicate e-mail for a second person	INSERT INTO person VALUES (9001,'Test','User','p1001@lender.example','LENDER')	IntegrityError: UNIQUE constraint failed: person.email
Identity: duplicate catalog number	INSERT INTO artifact VALUES (99999,'MM-0000001','Dup',1,1001,100)	IntegrityError: UNIQUE constraint failed: artifact.catalog_no
Identity: artifact whose current lender does not exist	INSERT INTO artifact VALUES (99998,'MM-NEW-1','Orphan',1,77777,100)	IntegrityError: FOREIGN KEY constraint failed
Lender subtype: lender row for a person that does not exist	INSERT INTO lender VALUES (88888,'INDIVIDUAL')	IntegrityError: FOREIGN KEY constraint failed

Workflow writes (dates)

Attempt	Statement	Actual message
Workflow: loan ends before it starts	INSERT INTO loan VALUES (9001,1001,6,'2026-05-10','2026-05-01',NULL,NULL)	IntegrityError: CHECK constraint failed: end_date > start_date

Attempt	Statement	Actual message
Workflow: returned before it started	INSERT INTO loan VALUES (9002,1001,6,'2026-05-10','2026-06-01','2026-05-01',NULL)	IntegrityError: CHECK constraint failed: returned_date IS NULL OR returned_date >= start_date

Category writes

Attempt	Statement	Actual message
Category: unknown employee role	INSERT INTO employee VALUES (1001,'JANITOR',1)	IntegrityError: CHECK constraint failed: emp_role IN ('CURATOR','REGISTRAR','OTHER')
Category: unknown condition grade	INSERT INTO condition_report VALUES (999999,1,(SELECT artifact_id FROM loan_object WHERE loan_id=1 LIMIT 1),'2026-01-05','Z',1)	IntegrityError: CHECK constraint failed: grade IN ('A','B','C','D')
Category: contact with both person and organization	INSERT INTO notification_contact VALUES (900,'PERSON',1001,1)	IntegrityError: CHECK constraint failed: (contact_type = 'PERSON' AND person_id IS NOT NULL AND org_id IS NULL) OR (contact_type = 'ORGANIZATION' AND org_id IS NOT NULL AND person_id IS NULL)
Category: contact with neither	INSERT INTO notification_contact VALUES (901,'ORGANIZATION',NULL,NULL)	IntegrityError: CHECK constraint failed: (contact_type = 'PERSON' AND person_id IS NOT NULL AND org_id IS NULL) OR (contact_type = 'ORGANIZATION' AND org_id IS NOT NULL AND person_id IS NULL)

Attempt	Statement	Actual message
Category: type says ORGANIZATION but person given	INSERT INTO notification_contact VALUES (902,'ORGANIZATION',1001,NULL)	IntegrityError: CHECK constraint failed: (contact_type = 'PERSON' AND person_id IS NOT NULL AND org_id IS NULL) OR (contact_type = 'ORGANIZATION' AND org_id IS NOT NULL AND person_id IS NULL)
Category: unknown contact type	INSERT INTO notification_contact VALUES (903,'AGENT',NULL,1)	IntegrityError: CHECK constraint failed: contact_type IN ('PERSON','ORGANIZATION')

Evidence, weak entity and history writes

Attempt	Statement	Actual message
Evidence: condition report for an artifact not in that loan	INSERT INTO condition_report VALUES (999998,1,1,'2026-01-05','A',1)	IntegrityError: FOREIGN KEY constraint failed
Weak entity: placement into a slot that does not exist	INSERT INTO placement SELECT lo.loan_id, lo.artifact_id, 1, 999 FROM loan_object lo LEFT JOIN placement p ON p.loan_id=lo.loan_id AND p.artifact_id=lo.artifact_id WHERE p.loan_id IS NULL LIMIT 1	IntegrityError: FOREIGN KEY constraint failed
Weak entity: slot number reused inside the same exhibition	INSERT INTO display_slot VALUES (1,1,'Gallery X')	IntegrityError: UNIQUE constraint failed: display_slot.exhibition_id, display_slot.slot_no
History: delete a loan that still has loan lines	DELETE FROM loan WHERE loan_id = 1	IntegrityError: FOREIGN KEY constraint failed

```
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQ
L server version for the right syntax to use near '6INSERT INTO person
VALUES (9002, 'Test', 'User',
        'p1001@lender.example'' at line 1
mysql> INSERT INTO person
    -> VALUES (9002, 'Test', 'User',
    ->         'p1001@lender.example', 'LENDER');
ERROR 1062 (23000): Duplicate entry 'p1001@lender.example' for key 'person.email'
mysql> INSERT INTO person
    -> VALUES (9002, 'Test', 'User',
    ->         'p1001@lender.example', 'LENDER');
ERROR 1062 (23000): Duplicate entry 'p1001@lender.example' for key 'person.email'
mysql>
```

```
    ->         'p1001@lender.example', 'LENDER');
ERROR 1062 (23000): Duplicate entry 'p1001@lender.example' for key 'person.email'
mysql> INSERT INTO lender
    -> VALUES (88888, 'INDIVIDUAL');
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (`museum_db`.`lende
r`, CONSTRAINT `lender_ibfk_1` FOREIGN KEY (`person_id`) REFERENCES `person` (`person_id`))
mysql>
```

```
mysql> INSERT INTO artifact
    -> VALUES (99999, 'MM-0000001', 'Duplicate Test', 1, 1001, 100);
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (`museum_db`.`artif
act`, CONSTRAINT `artifact_ibfk_2` FOREIGN KEY (`current_lender_id`) REFERENCES `lender` (`person_id`))
mysql>
```

```
mysql> INSERT INTO artifact
    -> VALUES (99998, 'MM-NEW-1', 'Orphan Test', 1, 77777, 100);
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (`museum_db`.`artif
act`, CONSTRAINT `artifact_ibfk_2` FOREIGN KEY (`current_lender_id`) REFERENCES `lender` (`person_id`))
mysql>
```

```
mysql> INSERT INTO lender
    -> VALUES (88888, 'INDIVIDUAL');
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (`museum_db`.`lende
r`, CONSTRAINT `lender_ibfk_1` FOREIGN KEY (`person_id`) REFERENCES `person` (`person_id`))
mysql>
```

```
mysql> INSERT INTO loan
    -> (loan_id, lender_id, start_date, end_date, returned_date)
    -> VALUES
    -> (99001, 88888, '2026-10-20', '2026-10-10', NULL);
ERROR 1364 (HY000): Field 'curator_id' doesn't have a default value
mysql>
```

```
    -> INSERT INTO employee
    -> (employee_id, person_id, unit_id, role)
    -> VALUES
    -> (99003, 9001, 1, 'INVALID_ROLE');
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQ
L server version for the right syntax to use near '13
INSERT INTO employee
(employee_id, person_id, unit_id, role)
VALUES
(99003, 9' at line 1
mysql>
```

```
ysql> INSERT INTO condition_report
    -> (loan_id, artifact_id, employee_id, condition_date, condition_grade)
    -> VALUES
    -> (99001, 99999, 99003, '2026-10-31', 'INVALID');
ERROR 1054 (42S22): Unknown column 'employee_id' in 'field list'
ysql>
```

8.5 Rules that need more than a row-level CHECK

Some rules involve two tables or two rows, so a CHECK (which sees one row) cannot enforce them. To prove this, each violation was inserted into a copy of the database (and then rolled back). The statement was **accepted** by the DBMS in every case, and only the detection query found it:

Table 20. Cross-table rules accepted by the schema

Business rule	Violating statement(s)	Result
Registrar (role REGISTRAR) recorded as handling curator	INSERT INTO loan VALUES (9100,1084,1,'2026-08-01','2026-12-01',NULL,NULL)	ACCEPTED, detection query returned: [9100,"REGISTRAR"]
Curator handles a loan of his or her own private object	INSERT INTO loan VALUES (9101,12,12,'2026-08-01','2026-12-01',NULL,NULL)	ACCEPTED, detection query returned: [9101,12,12]
Same artifact in a second loan with overlapping dates	INSERT INTO loan VALUES (9102,1084,6,'2025-01-08','2025-06-19',NULL,NULL) INSERT INTO loan_object VALUES (9102,8,1000)	ACCEPTED, detection query returned: [1,9102,8]

Business rule	Violating statement(s)	Result
Placement in an exhibition that runs outside the loan period	INSERT INTO exhibition VALUES (9103,'Late Show','2030-01-01','2030-02-01') INSERT INTO display_slot VALUES (9103,1,'Gallery 1') INSERT INTO placement SELECT lo.loan_id, lo.artifact_id, 9103, 1 FROM loan_object lo LEFT JOIN placement p ON p.loan_id=lo.loan_id AND p.artifact_id=lo.artifact_id WHERE p.loan_id IS NULL LIMIT 1	ACCEPTED, detection query returned: [2,729,9103]
Loan line for an artifact whose current lender is someone else	INSERT INTO loan_object VALUES (1,1,500)	ACCEPTED, detection query returned: [1,1,1084,1098]

These five rules need a trigger, a composite foreign key trick (for example a hidden role column that must equal CURATOR), a transaction-level check, or application logic. This is stated as a limitation in Task 11. Detection queries Q5, Q6 and Q7 in Task 9 are the safety net.

Task 9 - Validating the Design with Use-Case Queries

3 marks

Eight queries were executed one by one on all four populations. For each query the SQL and the actual output are shown for the 100-lender database. Output is limited to the first rows (the full counts for every size are in section 9.9). An empty result is reported as such.

9.1 Q1 Overdue loans still open

Which loans have passed their end date and are still open (returned_date is empty)? Overdue notices.

```
SELECT l.loan_id, l.lender_id, l.end_date
FROM loan l
WHERE l.returned_date IS NULL AND l.end_date < '2026-10-31'
ORDER BY l.end_date, l.loan_id
```

Actual output (N = 100): 4 rows, first 4 shown.

loan_id	lender_id	end_date
74	1019	2025-12-19
111	1083	2026-06-13
98	1011	2026-06-20
106	1019	2026-06-22

```
mysql> 25
    -> SELECT l.loan_id, l.lender_id, l.end_date
    -> FROM loan l
    -> WHERE l.returned_date IS NULL
    ->   AND l.end_date < '2026-10-31'
    -> ORDER BY l.end_date, l.loan_id;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQ
L server version for the right syntax to use near '25
SELECT l.loan_id, l.lender_id, l.end_date
FROM loan l
WHERE l.returned_date I' at line 1
mysql> SELECT l.loan_id, l.lender_id, l.end_date
    -> FROM loan l
    -> WHERE l.returned_date IS NULL
    ->   AND l.end_date < '2026-10-31'
    -> ORDER BY l.end_date, l.loan_id;
```

9.2 Q2 Loaned artifacts with no condition report

Which loaned artifacts have no condition report at all? This is the missing-evidence query.

```
SELECT DISTINCT lo.artifact_id, a.catalog_no
```

```
FROM loan_object lo
JOIN artifact a ON a.artifact_id = lo.artifact_id
LEFT JOIN condition_report cr ON cr.loan_id = lo.loan_id AND cr.artifact_id =
lo.artifact_id
WHERE cr.report_id IS NULL
ORDER BY lo.artifact_id
```

Actual output (N = 100): 67 rows, first 5 shown.

artifact_id	catalog_no
39	MM-0000039
53	MM-0000053
69	MM-0000069
70	MM-0000070
81	MM-0000081

```
mysql> 26
    -> SELECT DISTINCT lo.artifact_id, a.catalog_no
    -> FROM loan_object lo
    -> JOIN artifact a ON a.artifact_id = lo.artifact_id
    -> LEFT JOIN condition_report cr
    ->   ON cr.loan_id = lo.loan_id
    ->  AND cr.artifact_id = lo.artifact_id
    -> WHERE cr.report_id IS NULL
    -> ORDER BY lo.artifact_id;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQ
L server version for the right syntax to use near '26
SELECT DISTINCT lo.artifact_id, a.catalog_no
FROM loan_object lo
```

9.3 Q3 Artifacts, count of loans (successive loans)

Which artifacts have had successive loans? Shows that custody periods of one object are separate rows.

```
SELECT a.artifact_id, COUNT(*) AS loans
FROM artifact a JOIN loan_object lo ON lo.artifact_id = a.artifact_id
GROUP BY a.artifact_id HAVING COUNT(*) > 1
```

Actual output (N = 100): 79 rows, first 5 shown.

artifact_id	loans
20	2
39	3
41	2
44	2
53	2

```
mysql> 27
    -> SELECT a.artifact_id, COUNT(*) AS loans
    -> FROM artifact a
```

9.4 Q4 Exhibition inventory with lender

For each exhibition, how many objects are placed and how many different lenders are involved? Exhibition inventory through placements.

```
SELECT e.exhibition_id, COUNT(*) AS placed_objects, COUNT(DISTINCT l.lender_id)
AS lenders
FROM exhibition e
JOIN placement p ON p.exhibition_id = e.exhibition_id
JOIN loan l ON l.loan_id = p.loan_id
GROUP BY e.exhibition_id ORDER BY e.exhibition_id
```

Actual output (N = 100): 12 rows, first 5 shown.

exhibition_id	placed_objects	lenders
1	20	15
2	26	16
3	17	12
4	24	18
5	17	13

```
mysql> 28
    -> SELECT e.exhibition_id,
    ->        COUNT(*) AS placed_objects,
    ->        COUNT(DISTINCT l.lender_id) AS lenders
    -> FROM exhibition e
    -> JOIN placement p ON p.exhibition_id = e.exhibition_id
    -> JOIN loan l ON l.loan_id = p.loan_id
    -> GROUP BY e.exhibition_id
    -> ORDER BY e.exhibition_id;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQ
L server version for the right syntax to use near '28
SELECT e.exhibition_id,
       COUNT(*) AS placed_objects,
       COUNT(DISTI' at line 1
```

9.5 Q5 Overlapping custody periods for one artifact

Does any artifact appear in two loans whose periods overlap? Custody rule check.

```
SELECT a.loan_id AS loan_a, b.loan_id AS loan_b, a.artifact_id
FROM loan_object a
JOIN loan_object b ON a.artifact_id = b.artifact_id AND a.loan_id < b.loan_id
JOIN loan la ON la.loan_id = a.loan_id
JOIN loan lb ON lb.loan_id = b.loan_id
WHERE la.start_date < lb.end_date AND lb.start_date < la.end_date
```

Actual output (N = 100): empty result, 0 rows. No row meets the condition in the generated data, which is what the generator guarantees. It does not prove the schema prevents it (see Task 8.5).

```
mysql> 29
    -> SELECT a.loan_id AS loan_a,
    ->        b.loan_id AS loan_b,
    ->        a.artifact_id
    -> FROM loan_object a
    -> JOIN loan_object b
    ->   ON a.artifact_id = b.artifact_id
    ->  AND a.loan_id < b.loan_id
    -> JOIN loan la ON la.loan_id = a.loan_id
    -> JOIN loan lb ON lb.loan_id = b.loan_id
    -> WHERE la.start_date < lb.end_date
    ->   AND lb.start_date < la.end_date;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQ
L server version for the right syntax to use near '29
SELECT a.loan_id AS loan_a,
       b.loan_id AS loan_b,
       a.artifact_id
' at line 1
```

9.6 Q6 Placement outside loan period (exhibition not inside loan)

Is any object placed in an exhibition that is not inside its loan period? Links loan expiry to display slots.

```
SELECT p.loan_id, p.artifact_id, e.exhibition_id
```

```
FROM placement p
JOIN loan l ON l.loan_id = p.loan_id
JOIN exhibition e ON e.exhibition_id = p.exhibition_id
WHERE e.open_date < l.start_date OR e.close_date > l.end_date
```

Actual output (N = 100): empty result, 0 rows. No row meets the condition in the generated data, which is what the generator guarantees. It does not prove the schema prevents it (see Task 8.5).

```
mysql> 31
    -> SELECT lo.loan_id,
    ->        lo.artifact_id,
    ->        l.lender_id,
    ->        a.current_lender_id
    -> FROM loan_object lo
    -> JOIN loan l ON l.loan_id = lo.loan_id
    -> JOIN artifact a ON a.artifact_id = lo.artifact_id
    -> WHERE l.lender_id <> a.current_lender_id;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQ
L server version for the right syntax to use near '31
SELECT lo.loan_id,
       lo.artifact_id,
       l.lender_id,
       a.curren' at line 1
```

9.7 Q7 Loan lender differs from artifact's current lender

Is any loan line for an artifact whose current lender differs from the loan lender? Ownership versus custody check.

```
SELECT lo.loan_id, lo.artifact_id, l.lender_id, a.current_lender_id
FROM loan_object lo
JOIN loan l ON l.loan_id = lo.loan_id
JOIN artifact a ON a.artifact_id = lo.artifact_id
WHERE l.lender_id <> a.current_lender_id
```

Actual output (N = 100): empty result, 0 rows. No row meets the condition in the generated data, which is what the generator guarantees. It does not prove the schema prevents

it (see Task 8.5).

```
       a.curren' at line 1
mysql> 3
    -> SELECT p.person_id,
    ->        p.family_name,
    ->        (SELECT COUNT(*)
    ->         FROM loan l
    ->         WHERE l.curator_id = p.person_id) AS loans_handled,
    ->        (SELECT COUNT(*)
    ->         FROM loan l2
    ->         WHERE l2.lender_id = p.person_id) AS loans_lent
    -> FROM person p
    -> JOIN employee e ON e.person_id = p.person_id
    -> JOIN lender ld ON ld.person_id = p.person_id
    -> ORDER BY p.person_id;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQ
L server version for the right syntax to use near '3
SELECT p.person_id,
       p.family_name,
       (SELECT COUNT(*)
        FROM' at line 1
```

9.8 Q8 Curators who also lend, with loans handled

Which curators are also lenders, and how many loans do they handle and lend? Role overlap.

```
SELECT p.person_id, p.family_name,
    (SELECT COUNT(*) FROM loan l WHERE l.curator_id = p.person_id) AS
loans_handled,
    (SELECT COUNT(*) FROM loan l2 WHERE l2.lender_id = p.person_id) AS
loans_lent
FROM person p
JOIN employee e ON e.person_id = p.person_id
JOIN lender ld ON ld.person_id = p.person_id
ORDER BY p.person_id
```

Actual output (N = 100): 3 rows, first 3 shown.

person_id	family_name	loans_handled	loans_lent
12	Bose	5	3
17	Das	5	1
36	Joshi	4	0

9.9 Result counts, JOIN counts and median elapsed time

Each query was run 5 times per population and the median is reported. "JOINs" is the number of explicit JOIN keywords in the statement.

Table 21. Actual result counts

Query	JOINs	Rows N=100	Rows N=1k	Rows N=10k	Rows N=100k
Q1 Overdue loans still	0	4	60	703	6,695
Q2 Loaned artifacts with	2	67	636	6,432	64,045
Q3 Artifacts, count of	1	79	836	8,327	83,698
Q4 Exhibition inventory with	2	12	73	669	6,641
Q5 Overlapping custody periods	4	0	0	0	0
Q6 Placement outside loan	2	0	0	0	0
Q7 Loan lender differs	2	0	0	0	0
Q8 Curators who also	2	3	3	3	3

Table 22. Median elapsed time over 5 runs (SQLite, this environment)

Query	N=100 (ms)	N=1k (ms)	N=10k (ms)	N=100k (ms)
Q1 Overdue loans still	0.04	0.41	4.04	46.17
Q2 Loaned artifacts with	0.31	6.65	49.08	1350.2
Q3 Artifacts, count of	0.25	3.71	21.05	245.72
Q4 Exhibition inventory with	0.14	2.04	17.71	295.4
Q5 Overlapping custody periods	0.24	3.63	32.11	1024.26
Q6 Placement outside loan	0.08	1.18	10.66	224.65
Q7 Loan lender differs	0.11	1.53	33.52	765.82
Q8 Curators who also	0.05	0.35	2.19	39.24

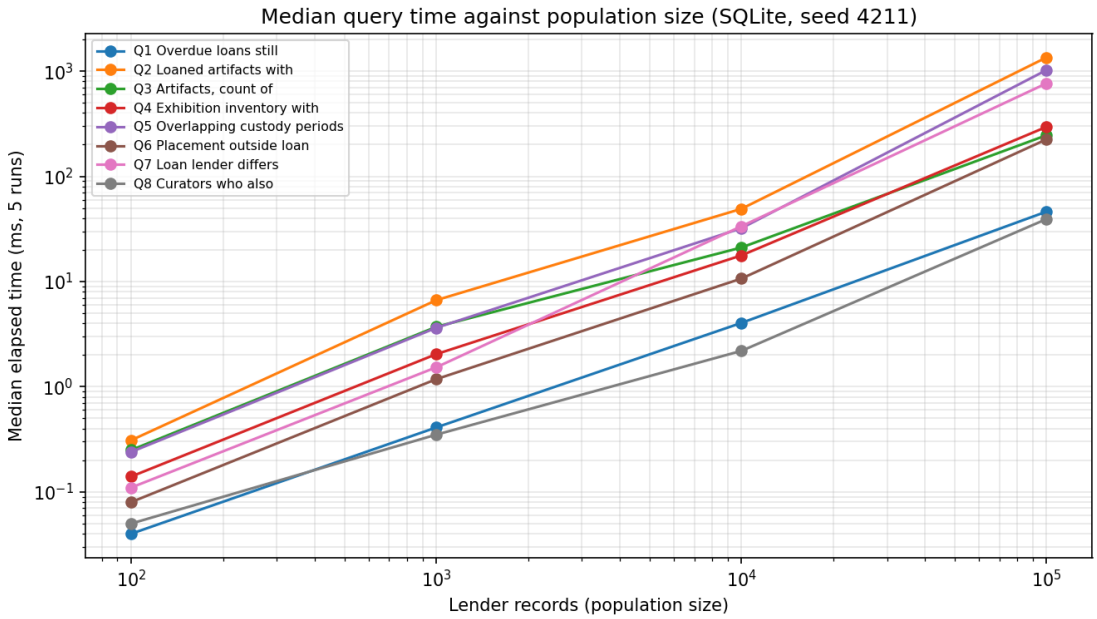


Figure 9. Median query time against population size (log-log)

Reading the timings: Q1 uses the index on end_date and scales roughly with the number of matching rows (46.17 ms at 100,000 lenders). The anti-join in Q2 and the self-join in Q5 are the most expensive (1350.2 ms and 1024.26 ms at 100,000). Growth is a little faster than linear for these two, which is expected for joins over about 442,000 loan lines. Absolute times depend on the machine and on SQLite; the shape is what matters.

Index evidence. For Q1 the query plan was captured with and without the index ix_loan_end:

Plan	Actual plan line
With index ix_loan_end	SEARCH loan USING INDEX ix_loan_end (end_date<?)
Without the index	SCAN loan

9.10 Flat LOAN file: insertion, update and deletion anomalies

To show why the design is split into tables, I loaded the same facts into one flat file flat_loan(loan_no, lender_name, lender_phone, curator_name, catalog_no, artifact_title, end_date, condition_note) and ran the operations. All outputs are actual.

Table 23. Initial flat file

loan_no	lender_name	lender_phone	curator_name	catalog_no	artifact_title	end_date	conditio
1	Raman Trust	044-1111	Divya Nair	MM-001	Bronze Lamp	2026-12-01	Minor sc
1	Raman Trust	044-1111	Divya Nair	MM-002	Silk Banner	2026-12-01	Good
2	Raman Trust	044-1111	Karthik Rao	MM-003	Stone Seal	2027-01-15	Good
3	Asha Iyer	98400-222	Divya Nair	MM-004	Clay Pot	2026-11-20	Hairline

Update anomaly. The lender Raman Trust changes its phone. The update was applied to one row only (loan 1, MM-001), as happens when someone edits one line of the register:

loan_no	lender_name	lender_phone
1	Raman Trust	044-9999
1	Raman Trust	044-1111
2	Raman Trust	044-1111

The same lender now has two different phone numbers (044-9999 and 044-1111) in three rows. In the normalised design the phone is stored once.

Deletion anomaly. Loan 3 is the only record of lender Asha Iyer and of the clay pot MM-004. After DELETE FROM flat_loan WHERE loan_no = 3 the query counting rows for that lender or artifact returned **0**. The lender and the artifact, and the condition note "Hairline crack", were lost with the loan.

Insertion anomaly. A new lender with no loan yet cannot be stored without inventing a loan. The insert was accepted only as a dummy row in which loan_no, catalog_no and every other column are NULL:

> Row accepted only with loan_no, catalog_no, etc. all NULL (dummy row): [(None, 'New Lender', '044-5555', None, None, None, None, None)]

In the normalised schema each of these facts (lender, artifact, loan, report) has its own table, so none of the three anomalies can occur.

Task 10 - Comparison with File-Based, Hierarchical and Network Models

4 marks

10.1 Access paths: tree nodes and network sets

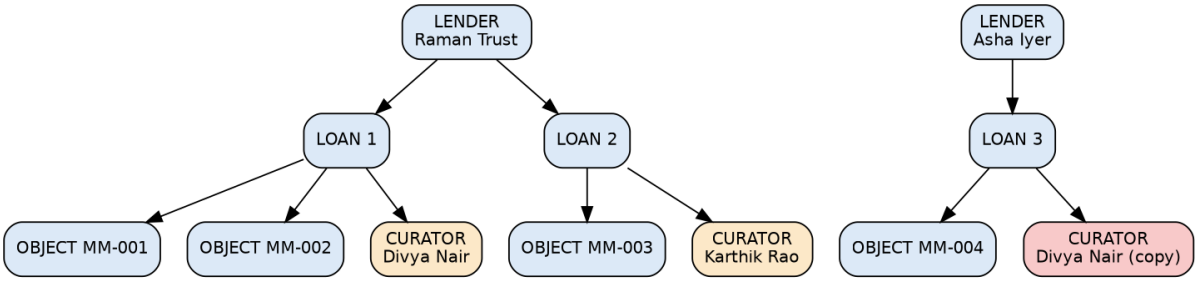


Figure 10. Hierarchical tree: LENDER at the root, LOAN and objects below; the curator must be copied under every loan

In a hierarchical model every record has exactly one parent. The natural root is LENDER, with LOAN below it and the loan objects below the loan. The handling curator is a second parent of LOAN, which a tree cannot express, so the curator record is **copied** under each loan (the red copy in the figure) or reached through a pointer outside the tree.

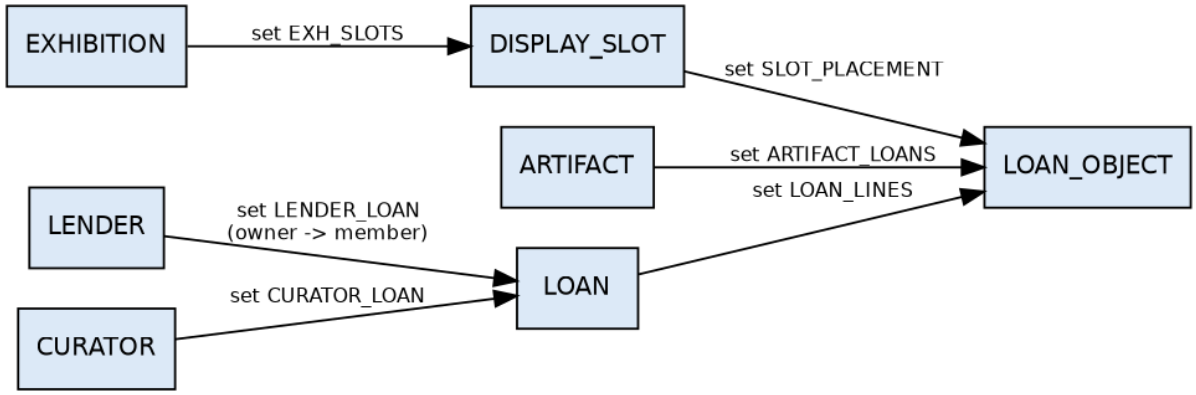


Figure 11. Network model: owner-member sets (arrow points from owner to member)

In a network (CODASYL-style) model a record can be a member of several sets, so LOAN is a member of LENDER_LOAN and CURATOR_LOAN at the same time. This removes the duplicate curator. The price is navigation: the program has to walk the sets record by record (find owner, find next member).

10.2 One person in several LOAN occurrences

Table 24. One person in several loans

Model	What happens when person P appears in several loans (as lender and as curator)
File-based	P's name and phone are repeated in every loan record of every file. A phone change must be made in each place (the update anomaly shown in Task 9.10). The register and the gallery plan are separate files and can disagree.
Hierarchical	As a lender P is one root with several LOAN children. As a curator P is repeated under every loan. Copies drift apart, and a loan whose lender is deleted disappears with the whole subtree, including its condition reports.
Network	P is one record that owns a LENDER_LOAN set and a CURATOR_LOAN set; every loan is a member of both. No copies. But a curator who is also a lender is still two records (a LENDER record and a CURATOR record), unless the schema is changed.
Relational (chosen)	P is one row in person; loan.lender_id and loan.curator_id both hold P's person_id. Roles overlap without copies, and the loan survives independent of P's other roles.

10.3 Tracing two case queries through each model

Table 25. Navigation of two queries

Query	File-based	Hierarchical	Network	Relational
Q1 overdue loans still open	Read every record of the loan file and test two dates in the program.	Visit every LENDER tree, then every LOAN child; there is no path from a date to a loan.	Scan all LOAN records (or keep a sorted set on end_date that must be maintained by hand).	One range lookup on ix_loan_end; the optimiser chooses the path (plan in Task 9.9).

Query	File-based	Hierarchical	Network	Relational
Q4 exhibition inventory with lenders	Match the exhibition, slot, placement and loan files in program loops.	EXHIBITION -> SLOT is a tree, but objects hang under LOAN, a different tree. Needs a second hierarchy or pointers.	EXHIBITION --EXH_SLOTS--> SLOT --SLOT_PLACEMENT--> LOAN_OBJECT, then up LOAN_LINES to LOAN and up LENDER_LOAN to LENDER, step by step.	Join exhibition, placement and loan on their keys; the DBMS plans the join order.

10.4 Views, conceptual model and internal representation (three-schema architecture)

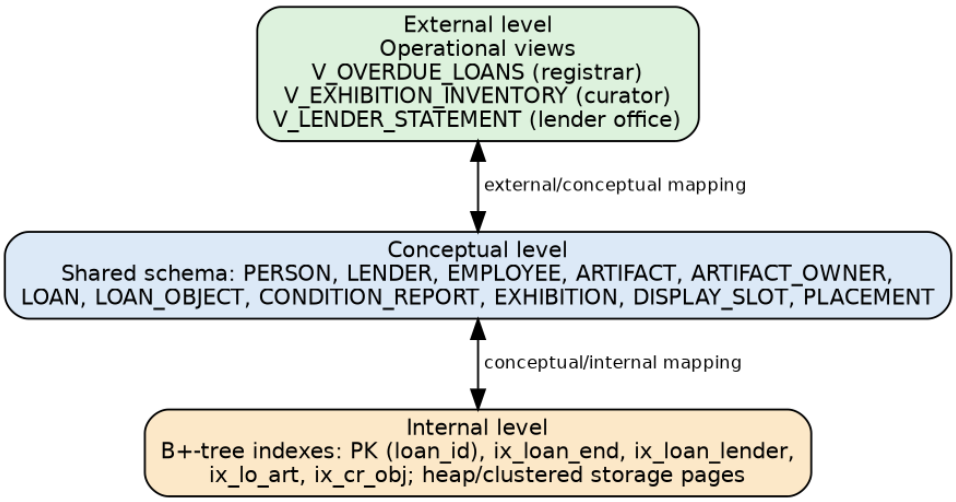


Figure 12. Three-schema architecture for Meridian Museum

- **Operational (external) view.** What one group sees. For the registrar the overdue list: CREATE VIEW v_overdue_loans AS SELECT loan_id, lender_id, end_date, days_overdue ... WHERE returned_date IS NULL AND end_date < CURRENT_DATE. It hides the other tables and columns.
- **Shared conceptual model.** The one schema of Task 7 that all views are defined on: PERSON, ARTIFACT, LOAN, LOAN_OBJECT, CONDITION_REPORT, EXHIBITION, DISPLAY_SLOT, PLACEMENT.
- **Indexed internal representation.** How rows are stored: clustered B+-tree on the primary key (loan_id), secondary B+-tree indexes ix_loan_end, ix_loan_lender,

ix_lo_art and ix_cr_obj, and pages in the buffer pool. Adding or dropping an index changes this level only; the view and the schema do not change.

Actual output of the overdue view on the 100-lender database (4 rows in total; today = 2026-10-31):

loan_id	lender_id	end_date	days_overdue
74	1019	2025-12-19	316
111	1083	2026-06-13	140
98	1011	2026-06-20	133
106	1019	2026-06-22	131

10.5 DBMS components in a concurrent update

Scenario on one loan record (loan 74): the registrar records returned_date while the loan administrator changes end_date at the same moment.

Table 26. Components involved in a concurrent update

Step	Component	What it does here
1	Query processor	Parses and plans both UPDATE statements and chooses the primary key lookup on loan_id = 74.
2	Transaction manager	Starts one transaction per user so each update is atomic and isolated.
3	Lock manager	The first transaction takes an exclusive row lock on loan 74. The second waits for it instead of overwriting.
4	Buffer manager and storage manager	Read the page of loan 74; the index entry in ix_loan_end is updated when end_date changes.
5	Log / recovery manager	Writes the change to the log before the page is changed, so a crash cannot leave half an update.
6	Transaction manager	On commit the lock is released; the second update then runs on the committed row and both changes survive.

Without the lock manager the second update could overwrite the first (a lost update), and the loan would be back to "open" even though the registrar had just closed it. This ties directly to custody periods: end_date and returned_date together define whether the object is still in custody.

Task 11 - Recommending and Defending the Final Design

4 marks

11.1 Recommendation

I recommend **Alternative C**: an independent LOAN with LOAN_OBJECT lines, evidence and placement attached to the line, and one shared PERSON identity with overlapping roles. Ownership history is kept out of the custody tables and is added later as its own association (Task 12).

Why it best preserves custody periods and condition evidence: every custody period is exactly one LOAN row, every report is tied to one (loan, artifact) pair, and neither depends on who owns the object. A looks cheaper but cannot reference a loan occurrence. B works for custody and evidence but keeps duplicate people. C removes that duplicate and costs only one more join.

11.2 Requirement-to-design trace

Table 27. Requirements traced to design elements

Requirement from the case	Entity / key / association	Evidence
One description per artifact	ARTIFACT(artifact_id), UNIQUE catalog_no	Duplicate catalog number rejected (Task 8.4)
Loan has one lender, many artifacts	LOAN(loan_id) with lender_id; LOAN_OBJECT(loan_id, artifact_id)	Loan 3 has four lines (Task 7.1)
Successive loans of one artifact	Several LOAN_OBJECT rows per artifact_id	Q3 returns artifacts with several loans
Reports belong to the object, not the lender	condition_report -> loan_object composite FK	Report for an artifact not in the loan rejected
Loan expiry linked to slots	placement -> loan_object and -> display_slot	Q6 empty; probe 4 shows the rule is not enforced by structure
Slot numbers within an exhibition	display_slot(exhibition_id, slot_no)	Missing slot rejected; slot reuse rejected

Requirement from the case	Entity / key / association	Evidence
Curator lends privately owned object	PERSON with roles; lender.person_id = employee.person_id	Q8 returns 3 curators who also lend
Ownership separate from custody	ARTIFACT_OWNER (Task 12)	Custody tables unchanged by migration

11.3 Tested results versus untested assertions

Table 28. What was tested

Tested (actual execution)	Not tested (business assertion only)
All 8 queries on 4 populations; row counts and timings	That registrars really file a report at intake and at return (the schema allows zero reports)
Constraint rejections: duplicate keys, bad dates, bad categories, missing parents, composite foreign keys, delete of a loan with lines	That the notification contact is always reachable
Cross-table rules accepted by the schema (5 probes)	That catalog_no never changes after cataloguing
Zero violations in Q5, Q6 and Q7 on generated data	Behaviour under many concurrent users (only described in Task 10.5, not measured)

11.4 Remaining limitations

- **Cross-table rules** are not enforced by the schema: curator role, curator not equal to lender, non-overlapping custody periods, exhibition inside loan period, and loan lender equal to the current lender. Each of these was accepted in the probes. They need triggers or application checks. Q5, Q6 and Q7 detect them after the fact.

- **Identity assumptions.** Person identity depends on e-mail being unique and being used consistently; the same human could still be registered twice under two addresses. catalog_no is assumed to be stable.
- **One current lender.** The single current lender per artifact cannot represent shared ownership. This is the assumption removed in Task 12.

11.5 Growing to 100,000 lenders

At 100,000 lenders the generated database holds 750,000 artifacts, 150,000 loans, 442,363 loan lines and 375,939 condition reports. Counts grew 1,000 times from N = 100 to N = 100,000 (loans from 148 to 150,000), and the three slowest queries (Q2, Q5 and Q7) grew by a factor of roughly 4,000 to 7,000, a little worse than linear, as expected for join-heavy queries. For evidence queries this means Q2 (loaned artifacts with no report) takes about 1.4 s, which is acceptable for a nightly check but not for a screen that refreshes. Practical steps are a composite index (artifact_id, loan_id) on loan_object, a flag or summary table for "has report", or running Q5 per exhibition instead of for the whole museum. Primary keys, foreign keys and the weak-entity design need no change at this size.

Task 12 - Re-evaluating and Adapting the Design

8 marks

Changed requirement: artifacts may have shared legal ownership, ownership shares can change while an existing loan remains in force, and the custody record must stay distinct from ownership history.

Decision: MODIFY (extend). Keep the custody, evidence and placement structure; add a versioned ownership association; remove the single-current-lender assumption.

A. Impact analysis (2 marks)

- **Invalidated assumption:** "each artifact has one current lender" (A1 in the case analysis). It is stored as artifact.current_lender_id and it also silently decides who may lend the artifact. Now an artifact has several owners at once and their shares change over time.
- **Affected elements:** entity ARTIFACT (the lender column), the 1:N association between lender and artifact (it becomes M:N with attributes), the business key of "who owns this" and the temporal rule that shares add up to 100 on every date. The rule Q7 ("loan lender equals current lender") also changes.
- **Why the old identifier no longer describes the fact:** artifact_id alone no longer identifies a single owner. The fact is now "person P owns share S of artifact A from date D1 to D2", so the identifier must be (artifact_id, person_id, valid_from). Same artifact, same person, two different periods are two different facts, for example 100% until 1 Oct 2026, then 50%.
- **Trace through the three levels:** operational views that show "artifacts of lender L" must now read ownership rows; the conceptual relation artifact_owner is added; at the internal level an index on artifact_owner(artifact_id) is needed so the owners of one artifact are found without scanning. Custody views are not touched.

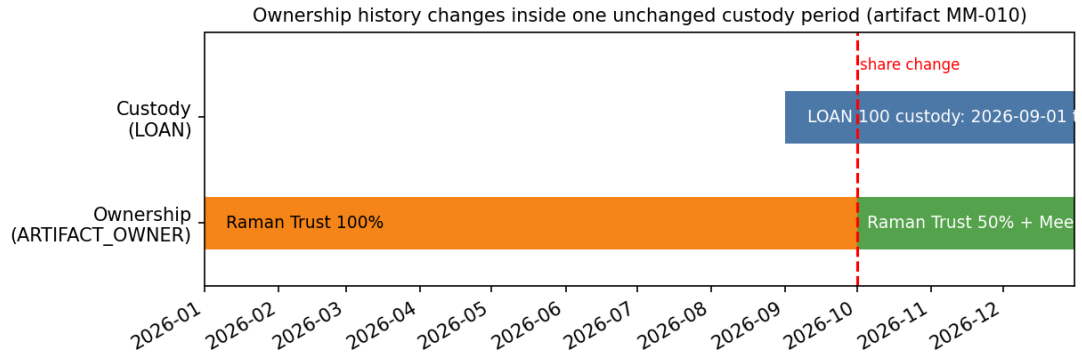


Figure 13. Ownership share changes inside one unchanged custody period

B. Evaluation of the original design (2 marks)

- **EXHIBITION representation:** exhibition, display_slot and placement refer only to loan_object, which refers to loan and artifact. None of them stores an owner, so the exhibition structure is not affected by an ownership change. Test: the migration leaves exhibition, display_slot and placement row counts unchanged at all four sizes (Task 12.D).
- **Stable primary keys:** artifact_id, loan_id, (loan_id, artifact_id), report_id, (exhibition_id, slot_no) and person_id stay the same. **Constraints that must change:** artifact.current_lender_id NOT NULL (an artifact has a set of owners, not one lender) and the Q7 rule, which becomes "the loan lender is an owner with share above 0 on the loan start date".
- **Rejected alternatives reconsidered:** Alternative A (history on a dated relationship) was weak for custody because loans could not be referenced. For **ownership** its idea fits well: an ownership share is exactly a dated M:N relationship with attributes (valid_from, valid_to, share_pct). So A's construct comes back, but only for ownership and not for custody. Alternative B fails here in the same way as C because both assume one current lender, and B would also need the PERSON change since owners can be curators or institutions. C remains the best base.

C. Adaptation (2 marks)

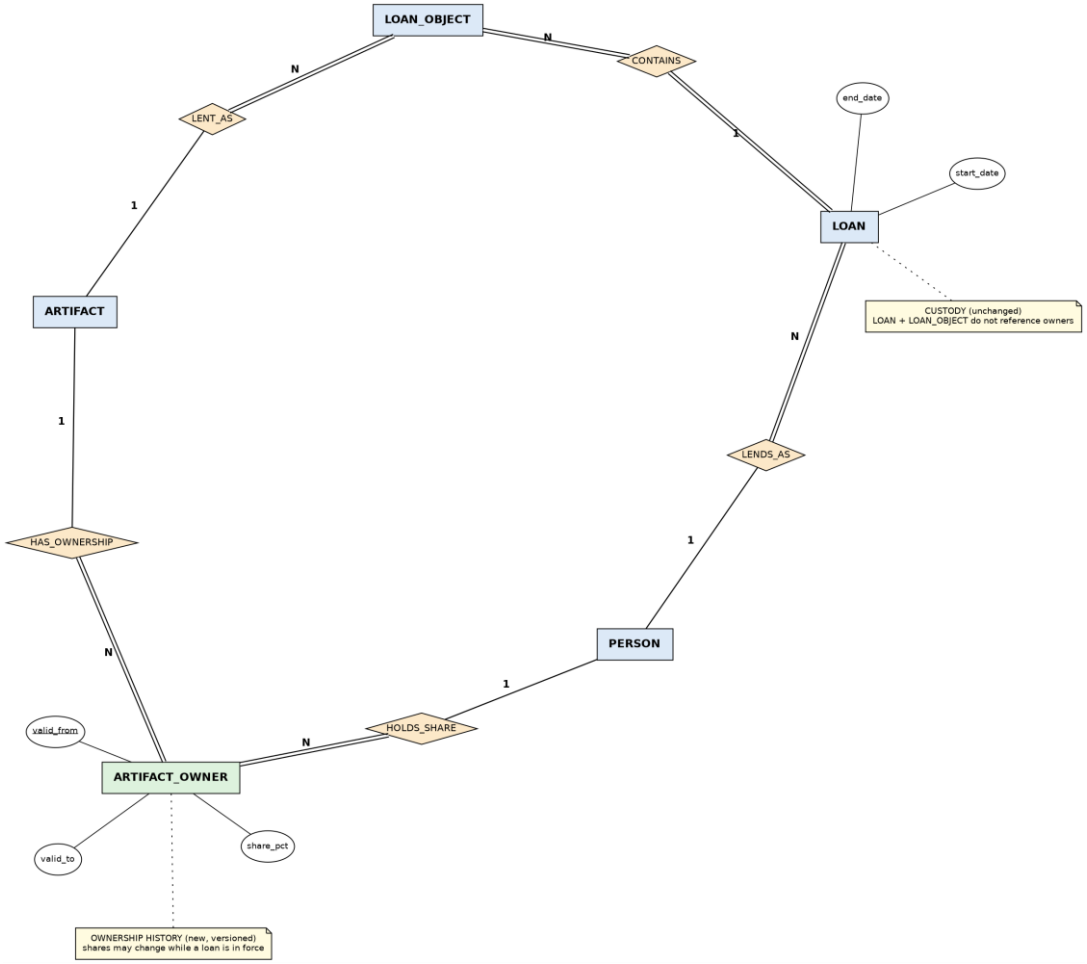


Figure 14. Revised EER: ARTIFACT_OWNER added; custody tables unchanged

Revised construct: ARTIFACT_OWNER is an associative entity between ARTIFACT and PERSON with attributes valid_from (partial key), valid_to and share_pct. Participation: every artifact has at least one current owner row (ARTIFACT 1..N), a person may own nothing (PERSON 0..N).

- **Replaced or extended:** the old association "CURRENT_LENDER" (1:N, one lender per artifact) is **replaced** by ARTIFACT_OWNER. The custody associations (MAKES, HANDLES, CONTAINS, LENT_AS, EVIDENCED_BY, PLACED_IN) are **kept** and not versioned, because custody already has its own period (LOAN start and end).

```
CREATE TABLE artifact_owner (
  artifact_id INT NOT NULL, person_id INT NOT NULL,
  valid_from  DATE NOT NULL, valid_to DATE NULL,
  share_pct   DECIMAL(5,2) NOT NULL,
  PRIMARY KEY (artifact_id, person_id, valid_from),
```

```
  CONSTRAINT ck_share  CHECK (share_pct > 0 AND share_pct <= 100),
  CONSTRAINT ck_valid  CHECK (valid_to IS NULL OR valid_to > valid_from),
  FOREIGN KEY (artifact_id) REFERENCES artifact(artifact_id),
  FOREIGN KEY (person_id)   REFERENCES person(person_id)
);
```

Migration that preserves historical evidence

9. Create artifact_owner (above). Do not touch loan, loan_object, condition_report or placement.
10. Copy the old single lender as a 100% owner: INSERT INTO artifact_owner SELECT artifact_id, current_lender_id, '2020-01-01', NULL, 100 FROM artifact; The date 2020-01-01 is the start of the museum register, because the real acquisition date is unknown; this is stated in the data dictionary.
11. Run the verification queries in section D. Only if they return zero rows, drop the old column or keep it as deprecated.
12. Record each share change as two steps: close the old row (set valid_to) and insert the new rows. Never update share_pct in place, or the history is lost.

Worked example (small, actual output). Artifact 10 (MM-010) is on loan 100 from 1 Sep to 31 Dec 2026. On 1 Oct 2026 half is sold to Meena Das while the loan is in force:

Table 29. Ownership rows for artifact 10 after the share change

artifact_id	person_id	valid_from	valid_to	share_pct
10	1	2020-01-01	2026-10-01	100
10	1	2026-10-01	NULL	50
10	3	2026-10-01	NULL	50

Table 30. Loan 100 after the share change (unchanged)

loan_id	lender_id	start_date	end_date
100	1	2026-09-01	2026-12-31

Owners on 2026-09-15: [[1,100]] (person 1 with 100%). Owners on 2026-10-15: [[1,50],[3,50]] (person 1 and person 3 with 50% each). The custody row did not change, and both ownership states can be asked for by date.

D. Evidence-based defence (2 marks)

New identity and participation rules

Identity: (artifact_id, person_id, valid_from). Participation: ARTIFACT (1,N) to ARTIFACT_OWNER, PERSON (0,N), and for every artifact and date the active shares must sum to 100. The share total cannot be a row-level CHECK, so it is verified by query.

Queries that would reveal an incorrect migration

```
-- 1. artifacts that lost their owner
SELECT a.artifact_id FROM artifact a
WHERE NOT EXISTS (SELECT 1 FROM artifact_owner o
                  WHERE o.artifact_id = a.artifact_id AND o.valid_to IS NULL);

-- 2. current shares that do not total 100
SELECT artifact_id, SUM(share_pct) AS total FROM artifact_owner
WHERE valid_to IS NULL GROUP BY artifact_id
HAVING ABS(SUM(share_pct) - 100.0) > 0.001;

-- 3. loan lender who was not an owner when the loan started
SELECT lo.loan_id, lo.artifact_id FROM loan_object lo JOIN loan l ON l.loan_id =
lo.loan_id
WHERE NOT EXISTS (SELECT 1 FROM artifact_owner o
  WHERE o.artifact_id = lo.artifact_id AND o.person_id = l.lender_id
    AND o.valid_from <= l.start_date AND (o.valid_to IS NULL OR o.valid_to >
l.start_date));
```

To prove query 2 works, I fed it a deliberately wrong migration (an owner given 100% twice, and an artifact with 50% + 40%). Actual output:

Table 31. Query 2 on a wrong migration (detects both mistakes)

artifact_id	total
11	200
13	90

Effect on generated data and relationship counts

Table 32. Migration run on all four populations (actual)

Lenders	Artifacts	Owner rows after copy	Artifacts with changed shares (20%)	Owner rows after change	Rows per artifact	Query 1 / 2 / 3 results
103	750	750	150	1,050	1.40	0 / 0 / 0
1,003	7,500	7,500	1,500	10,500	1.40	0 / 0 / 0
10,003	75,000	75,000	15,000	105,000	1.40	0 / 0 / 0
100,003	750,000	750,000	150,000	1,050,000	1.40	0 / 0 / 0

The ownership copy creates exactly one row per artifact (750,000 at 100,000 lenders). After 20% of the artifacts get a share change (seeded, 2026-10-01), the table holds 1.4 rows per artifact. Row counts of artifact, loan, loan_object, condition_report and placement were identical before and after in all four runs (checked: identical), so custody and evidence are untouched. All three verification queries returned **0 rows** at every size.

Keeping existing operational views working

Views that expect one lender per artifact can keep their interface by reading the owner with the largest current share:

```
CREATE VIEW artifact_current_lender AS
SELECT a.artifact_id, a.catalog_no,
  (SELECT person_id FROM artifact_owner o
    WHERE o.artifact_id = a.artifact_id AND o.valid_to IS NULL
    ORDER BY share_pct DESC, person_id LIMIT 1) AS current_lender_id
FROM artifact a;
```

Table 33. Compatibility view output on the worked example (artifact 12 has no owner and shows NULL, matching the "artifact without owner" check)

artifact_id	catalog_no	current_lender_id
10	MM-010	1
11	MM-011	2
12	MM-012	NULL

Cost of the extra association and history checks

Table 34. Median time over 5 runs (SQLite)

Lenders	Loan lines with direct column (rows, ms)	Loan lines joined to current owners (rows, ms)	Time ratio
100	423, 0.26	522, 0.42	1.6x
1,000	4,426, 3.32	5,359, 10.53	3.2x
10,000	44,214, 61	53,023, 154.38	2.5x
100,000	442,363, 941.74	530,304, 2083.56	2.2x

The join to current owners returns more rows (one per owner, 20% of the artifacts have two) and takes about 1.6 to 3.2 times as long as reading the single column. Storage grows by 40% in this table (1.4 rows per artifact). The three verification queries are run once after migration and then as a periodic check, not on every transaction.

Final decision related to the changed requirement

The new requirement says custody must stay distinct from ownership history. In this design the two are physically separate: loan, loan_object, condition_report and placement have no ownership column, and the migration changed none of their rows. Only the table that answers "who owns it, in what share, from when" is new. That is why the right response is to **modify** the model and not to redesign it: the keys and relations that carry custody periods and condition evidence stay the same, and the added cost is one association, one index and three verification queries.

Table 35. Constraints of the new association

Constraint test (actual messages)	Statement	Result
Ownership: share of 0 per cent	INSERT INTO artifact_owner VALUES (1,1001,'2026-11-01',NULL,0)	IntegrityError: CHECK constraint failed: share_pct > 0 AND share_pct <= 100
Ownership: share above 100	INSERT INTO artifact_owner VALUES (1,1001,'2026-11-01',NULL,120)	IntegrityError: CHECK constraint failed: share_pct > 0 AND share_pct <= 100
Ownership: valid_to before valid_from	INSERT INTO artifact_owner VALUES (1,1001,'2026-11-01','2026-10-01',50)	IntegrityError: CHECK constraint failed: valid_to IS NULL OR valid_to > valid_from
Ownership: owner is not a person	INSERT INTO artifact_owner VALUES (1,555555,'2026-11-01',NULL,50)	IntegrityError: FOREIGN KEY constraint failed
Ownership: same owner, same start twice	INSERT INTO artifact_owner SELECT artifact_id,person_id,valid_from,NULL,10 FROM artifact_owner LIMIT 1	IntegrityError: UNIQUE constraint failed: artifact_owner.artifact_id, artifact_owner.person_id, artifact_owner.valid_from

Appendix A - MySQL 8.0 DDL

Foreign keys use the default RESTRICT behaviour unless stated. InnoDB is required for foreign keys. CHECK constraints are enforced from MySQL 8.0.16.

```
CREATE TABLE collection_unit (
  unit_id TINYINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  unit_name VARCHAR(60) NOT NULL UNIQUE
) ENGINE=InnoDB;

CREATE TABLE person (
  person_id INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  given_name VARCHAR(60) NOT NULL, family_name VARCHAR(60) NOT NULL,
  email VARCHAR(120) NOT NULL UNIQUE,
  person_kind VARCHAR(8) NOT NULL,
  CONSTRAINT ck_kind CHECK (person_kind IN ('LENDER','EMPLOYEE','BOTH'))
) ENGINE=InnoDB AUTO_INCREMENT=1;

CREATE TABLE person_phone (
  person_id INT UNSIGNED NOT NULL, phone_no VARCHAR(20) NOT NULL,
  PRIMARY KEY (person_id, phone_no),
  FOREIGN KEY (person_id) REFERENCES person(person_id) ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE lender (
  person_id INT UNSIGNED PRIMARY KEY, lender_type VARCHAR(12) NOT NULL,
  CONSTRAINT ck_ltype CHECK (lender_type IN ('INDIVIDUAL','INSTITUTION')),
  FOREIGN KEY (person_id) REFERENCES person(person_id)
) ENGINE=InnoDB;

CREATE TABLE employee (
  person_id INT UNSIGNED PRIMARY KEY, emp_role VARCHAR(10) NOT NULL,
  unit_id TINYINT UNSIGNED NULL,
  CONSTRAINT ck_role CHECK (emp_role IN ('CURATOR','REGISTRAR','OTHER')),
  FOREIGN KEY (person_id) REFERENCES person(person_id),
  FOREIGN KEY (unit_id) REFERENCES collection_unit(unit_id)
```

```
) ENGINE=InnoDB;

CREATE TABLE artifact (
  artifact_id INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  catalog_no VARCHAR(20) NOT NULL UNIQUE, title VARCHAR(150) NOT NULL,
  unit_id TINYINT UNSIGNED NOT NULL, current_lender_id INT UNSIGNED NOT
NULL,
  declared_value DECIMAL(12,2) NOT NULL,
  CONSTRAINT ck_val CHECK (declared_value >= 0),
  FOREIGN KEY (unit_id) REFERENCES collection_unit(unit_id),
  FOREIGN KEY (current_lender_id) REFERENCES lender(person_id)
) ENGINE=InnoDB;

CREATE TABLE organization (
  org_id INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  org_name VARCHAR(120) NOT NULL UNIQUE
) ENGINE=InnoDB;

CREATE TABLE notification_contact (
  contact_id INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  contact_type VARCHAR(12) NOT NULL,
  person_id INT UNSIGNED NULL, org_id INT UNSIGNED NULL,
  CONSTRAINT ck_ctype CHECK (contact_type IN ('PERSON','ORGANIZATION')),
  CONSTRAINT ck_one_source CHECK (
    (contact_type='PERSON' AND person_id IS NOT NULL AND org_id IS NULL) OR
    (contact_type='ORGANIZATION' AND org_id IS NOT NULL AND person_id IS
NULL)),
  FOREIGN KEY (person_id) REFERENCES person(person_id),
  FOREIGN KEY (org_id) REFERENCES organization(org_id)
) ENGINE=InnoDB;

CREATE TABLE loan (
  loan_id INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  lender_id INT UNSIGNED NOT NULL, curator_id INT UNSIGNED NOT NULL,
```

```
  start_date DATE NOT NULL, end_date DATE NOT NULL, returned_date DATE NULL,
  notification_contact_id INT UNSIGNED NULL,
  CONSTRAINT ck_loan_period CHECK (end_date > start_date),
  CONSTRAINT ck_loan_returned CHECK (returned_date IS NULL OR returned_date >=
start_date),
  FOREIGN KEY (lender_id) REFERENCES lender(person_id),
  FOREIGN KEY (curator_id) REFERENCES employee(person_id),
  FOREIGN KEY (notification_contact_id) REFERENCES
notification_contact(contact_id),
  INDEX ix_loan_lender (lender_id), INDEX ix_loan_end (end_date)
) ENGINE=InnoDB;

CREATE TABLE loan_object (
  loan_id INT UNSIGNED NOT NULL, artifact_id INT UNSIGNED NOT NULL,
  loan_value DECIMAL(12,2) NOT NULL,
  PRIMARY KEY (loan_id, artifact_id),
  CONSTRAINT ck_lv CHECK (loan_value >= 0),
  FOREIGN KEY (loan_id) REFERENCES loan(loan_id),
  FOREIGN KEY (artifact_id) REFERENCES artifact(artifact_id),
  INDEX ix_lo_art (artifact_id)
) ENGINE=InnoDB;

CREATE TABLE condition_report (
  report_id INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  loan_id INT UNSIGNED NOT NULL, artifact_id INT UNSIGNED NOT NULL,
  report_date DATE NOT NULL, grade CHAR(1) NOT NULL, registrar_id INT
UNSIGNED NOT NULL,
  CONSTRAINT ck_grade CHECK (grade IN ('A','B','C','D')),
  FOREIGN KEY (loan_id, artifact_id) REFERENCES loan_object(loan_id, artifact_id),
  FOREIGN KEY (registrar_id) REFERENCES employee(person_id)
) ENGINE=InnoDB;

CREATE TABLE exhibition (
  exhibition_id INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
```

```
  exhibition_name VARCHAR(120) NOT NULL,
  open_date DATE NOT NULL, close_date DATE NOT NULL,
  CONSTRAINT ck_exh CHECK (close_date > open_date)
) ENGINE=InnoDB;

CREATE TABLE display_slot (
  exhibition_id INT UNSIGNED NOT NULL, slot_no SMALLINT UNSIGNED NOT
NULL,
  gallery VARCHAR(40) NOT NULL,
  PRIMARY KEY (exhibition_id, slot_no),
  CONSTRAINT ck_slot CHECK (slot_no > 0),
  FOREIGN KEY (exhibition_id) REFERENCES exhibition(exhibition_id) ON DELETE
CASCADE
) ENGINE=InnoDB;

CREATE TABLE placement (
  loan_id INT UNSIGNED NOT NULL, artifact_id INT UNSIGNED NOT NULL,
  exhibition_id INT UNSIGNED NOT NULL, slot_no SMALLINT UNSIGNED NOT
NULL,
  PRIMARY KEY (loan_id, artifact_id),
  UNIQUE KEY uq_slot (exhibition_id, slot_no),
  FOREIGN KEY (loan_id, artifact_id) REFERENCES loan_object(loan_id, artifact_id),
  FOREIGN KEY (exhibition_id, slot_no) REFERENCES display_slot(exhibition_id,
slot_no)
) ENGINE=InnoDB;

CREATE INDEX ix_cr_obj ON condition_report(loan_id, artifact_id);
```

Appendix B - Reproducing the run

13. Create the schema from Appendix A (or schema_sqlite.sql for the SQLite run).
14. Run python3 generate_and_run.py 100, then 1000, 10000 and 100000. Each call creates its own database, loads it with seed 4211, runs the eight queries five times and writes the counts and median times.
15. Run python3 tests.py for the invalid-write tests and the flat-file anomalies, python3 extras.py for the views and the cross-table probes, and python3 task12.py for the ownership migration on all four sizes.